# CẤU TRÚC DỮ LIỆU ĐA CHIỀU

Hệ cơ sở dữ liệu đa phương tiện

# Giới thiệu

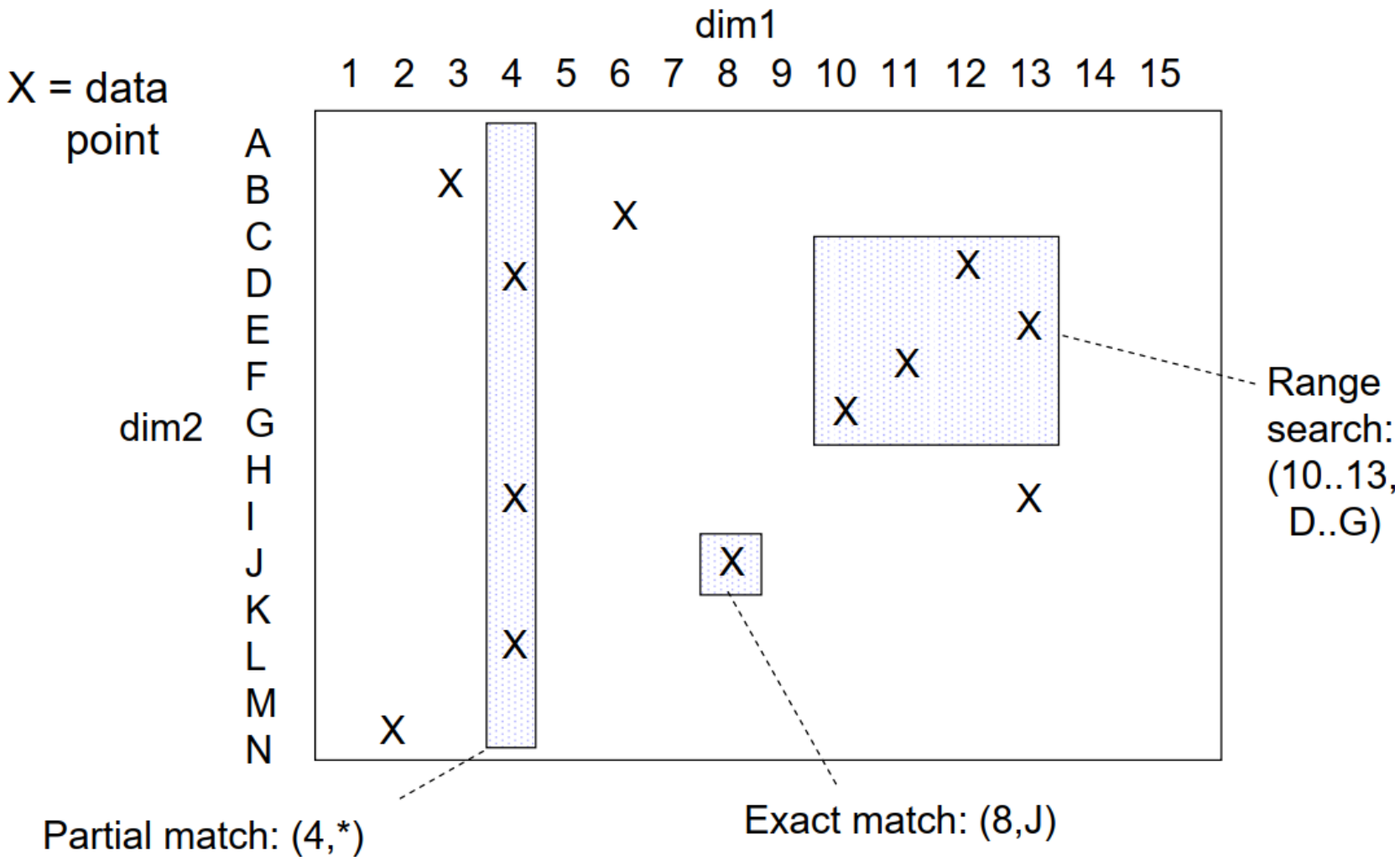
- Hai nhóm dữ liệu chính
  - **Dữ liệu điểm (Point data):** đối tượng csdl gồm *k*-tuple (bộ dữ liệu) trong không gian *k* chiều.
    - ✓Mỗi phần tử bộ dữ liệu tương ứng với tọa độ trong không gian
    - ✓Truy xuất đa thuộc tính từ CSDL quan hệ, CSDL văn bản, vector đặc trưng của các đối tượng đa phương tiện
  - **Dữ liệu không gian (Spatial data):** các đối tượng CSDL có một số loại hình dạng (shape) và kích thước (size) khác nhau
    - ✓Điểm là trường hợp đặc biệt
    - ✓Bản vẽ CAD, thiết kế VLSI, địa lý, xử lý ảnh…

# Giới thiệu

## ❖Yêu cầu cơ bản đối với cấu trúc dữ liệu đa chiều:

- ➢Cần đảm bảo việc sử dụng lưu trữ tốt
- ➢Những tác vụ (task) đơn giản chỉ yêu cầu truy cập đĩa ít
- ➢Các chiều khác nhau nên được xử lý theo một cách đối xứng
- ➢Phân cụm đối tượng phải tuân theo độ lân cận hình học
- ➢Phải cho phép tổ chức lại theo hướng "động" khi tập dữ liệu mở rộng hoặc thu hẹp
- ➢Giải thuật tìm kiếm và cập nhật phải đơn giản
- ➢Cấu trúc hỗ trợ các loại truy vấn khác nhau

# Các loại truy vấn trong không gian 02 chiều



# Các loại truy vấn đa chiều

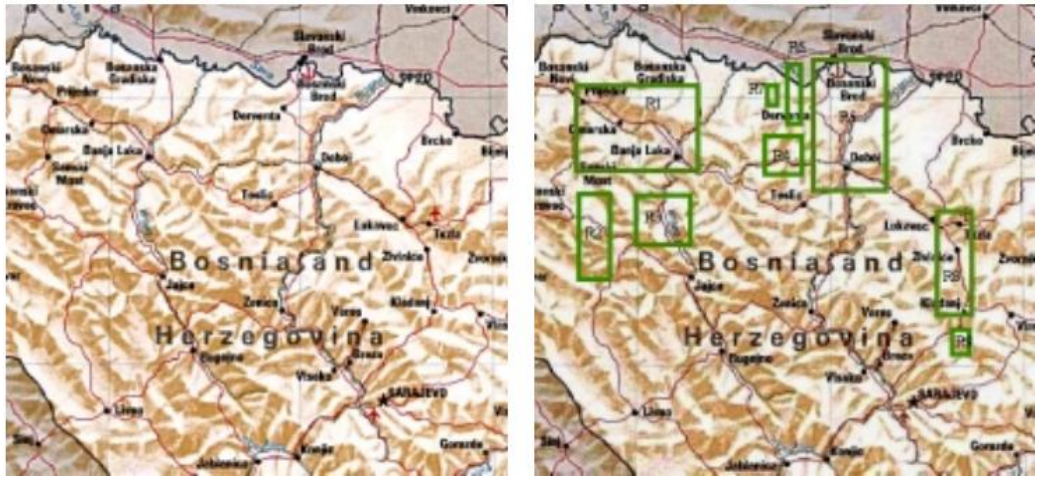
- Truy vấn đối sánh chính xác (exact-match queries):
  - Tất cả các tọa độ (thuộc tính) được cố định trong truy vấn. Độ phức tạp là hàm log
- Truy vấn đối sánh một phần (partial-match queries):
  - Chỉ $t$ trong $k$ tọa độ được xác định trong truy vấn. Phần còn lại có giá trị tùy ý
  - Cận dưới cho độ phức tạp trong trường hợp xấu nhất: $O(n^{1-\frac{t}{k}})$

# Các loại truy vấn đa chiều

- Truy vấn phạm vi (range queries):
  - Đối với mỗi chiều, một phạm vi giá trị được xác định
  - Đối sánh chính xác: phạm vi $= [-c, c]$; đối sánh một phần $= (-\infty, \infty)$ đối với mỗi tọa độ
- Truy vấn phù hợp nhất (best-match queries):
  - Tìm láng giềng gần nhất của điểm/khu vực được chỉ định bởi điều kiện truy vấn (chính xác hoặc phạm vi)
  - Tìm *k* láng giềng gần nhất: như trên
  - Truy vấn xếp hạng: *k* láng giềng gần nhất theo thứ tự lân cận

# Cấu trúc dữ liệu ĐPT





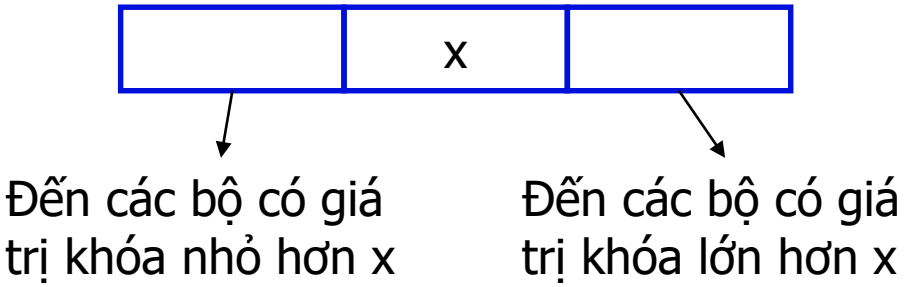

- Bản đồ có thể xem như hình ảnh 2 chiều
  - Point: là các vị trí cần quan tâm
- Các điểm được lưu trữ trong cấu trúc dữ liệu chuyên biệt”
  - k-d Tree
  - Point quadtree
  - MX-Quadtree
  - R-trees

# k-D trees

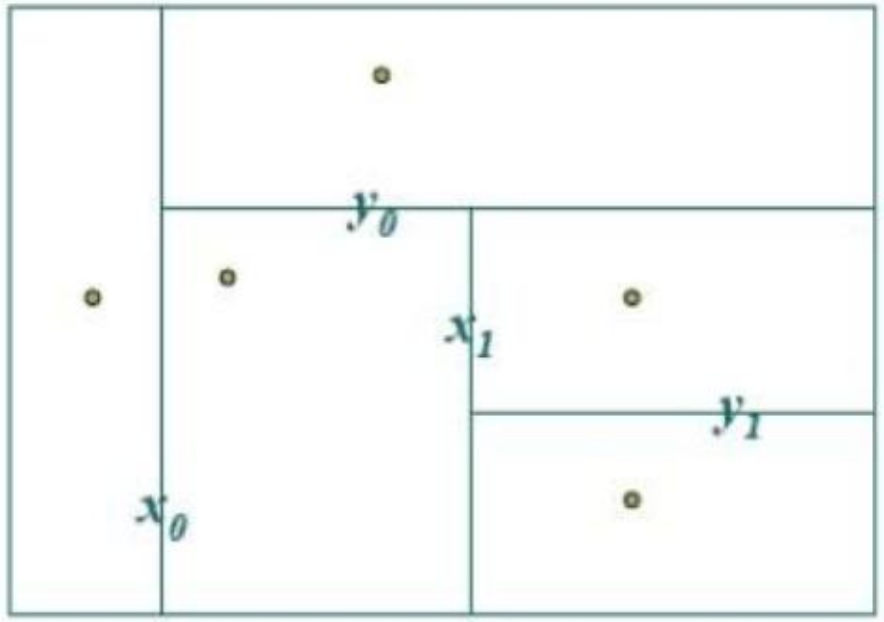
- Dành lưu trữ dữ liệu điểm đa chiều (k-dimensional point)
  - 2-tree: lưu dữ liệu điểm 02 chiều
  - 3-tree: lưu dữ liệu điểm 03 chiều
  - …
  - Mỗi điểm là vector có *k* phần tử
- Không lưu dữ liệu không gian (dữ liệu vùng)

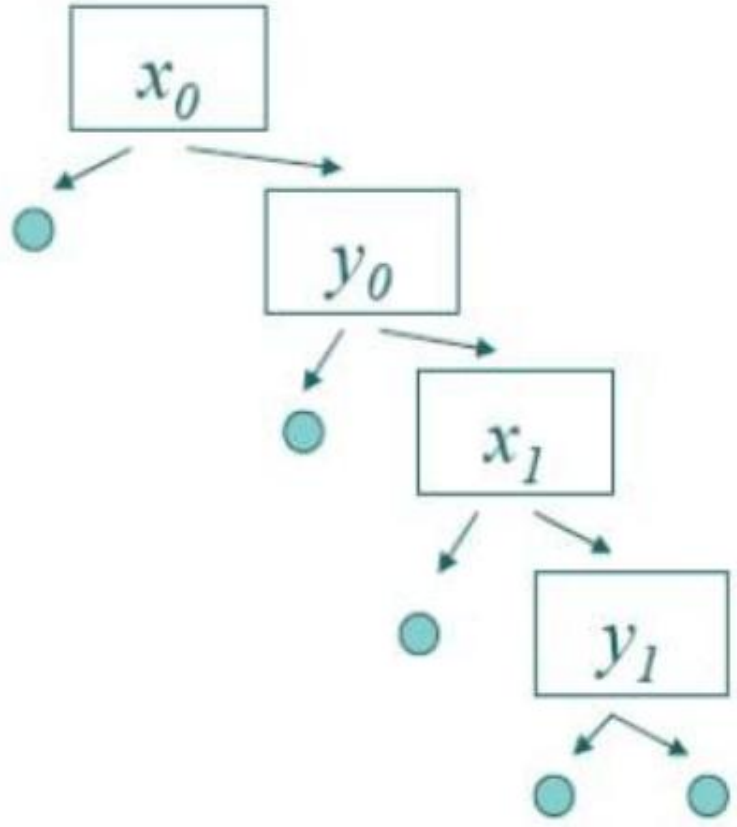
# k-D trees

- Là mở rộng của cây nhị phân
- Ở mỗi mức, các bản ghi sẽ được chia theo giá trị của 01 chiều nhất định:
  - Mức 0: giá trị chiều 0
  - …
  - Mức k-1: giá trị chiều k-1
  - Mức k: giá trị chiều 0



- Ví dụ: 2-D trees



# k-D trees

❖Cấu trúc 1 nút

| INFO | XVAL | YVAL |
|---|---|---|
| LLINK | RLINK | |

❖Định nghĩa: 2-D tree là cây nhị phân thỏa mãn:

➢Nếu nút $N$ ở mức chẵn:

$$\forall M \in N.LLINK: M.XVAL < N.XVAL \;\&$$
$$\forall P \in N.RLINK: P.XVAL \geq N.XVAL$$

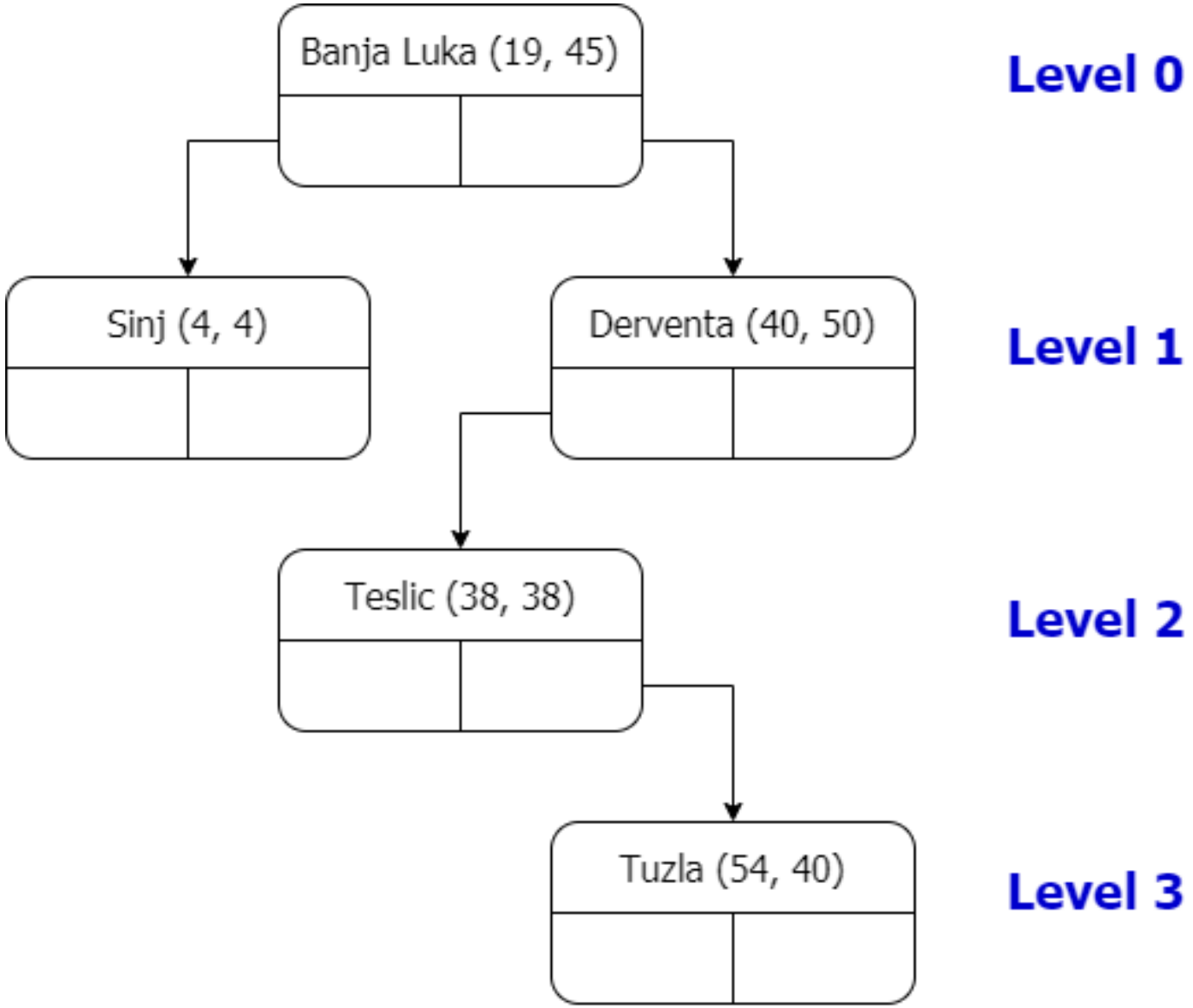
➢Nếu nút $N$ ở mức lẻ:

$$\forall M \in N.LLINK: M.YVAL < N.YVAL \;\&$$
$$\forall P \in N.RLINK: P.YVAL \geq N.YVAL$$

# k-D trees

- Ví dụ:
- Thứ tự insert:

| INFO | XVAL | YVAL |
|---|---|---|
| Banja Luka | 19 | 45 |
| Derventa | 40 | 50 |
| Teslic | 38 | 38 |
| Tuzla | 54 | 40 |
| Sinj | 4 | 4 |



# Chèn (insert)/ tìm kiếm (search)

- Nút cần thêm: $P(\text{info}, x, y)$
- Các bước (lặp lại cho đến khi kết thúc)
  - Nút đang duyệt: N
  - Nếu *N.XVAL = x và N.YVAL = y* thì ghi đè N và kết thúc
  - Nếu N ở mức chẵn (0, 2, 4, …)
    - ✓ Nếu *x < N.XVAL* thì duyệt cây con bên trái
    - ✓ Ngược lại, duyệt cây con bên phải
  - Nếu N ở mức lẻ (1, 3, 5…)
    - ✓ Nếu *y < N.YVAL* thì duyệt cây con bên trái
    - ✓ Ngược lại, duyệt cây con bên phải

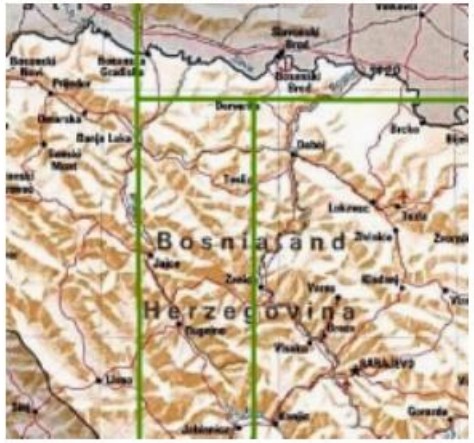
# Ví dụ của chèn



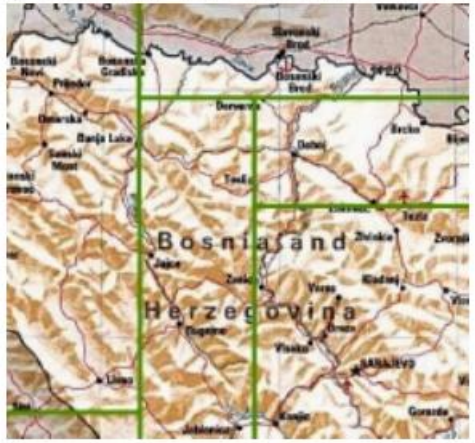

(a) Splitting of region by Banja Luka




(b) Splitting of region by Derventa

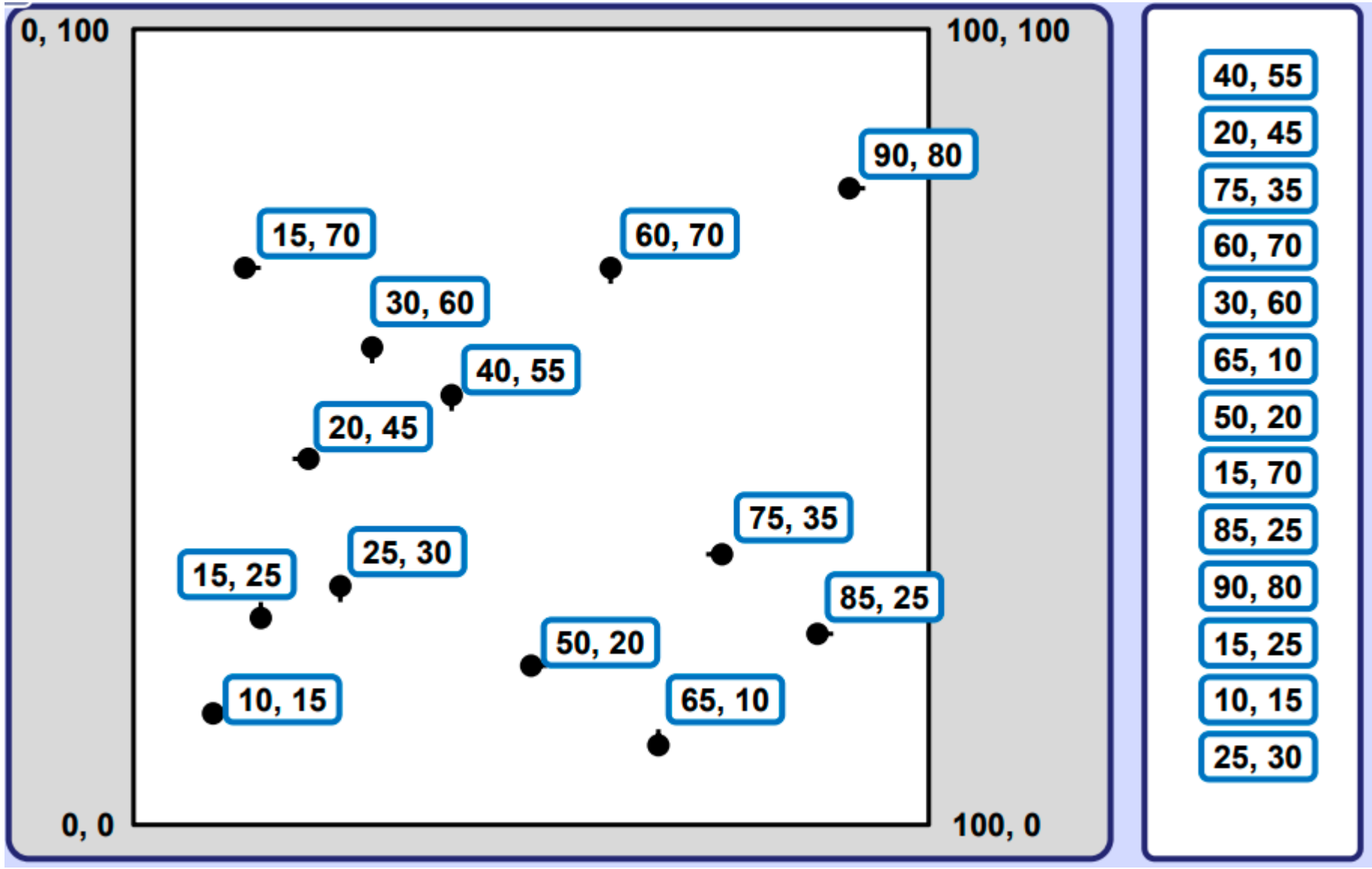



(c) Splitting of region by Toslic

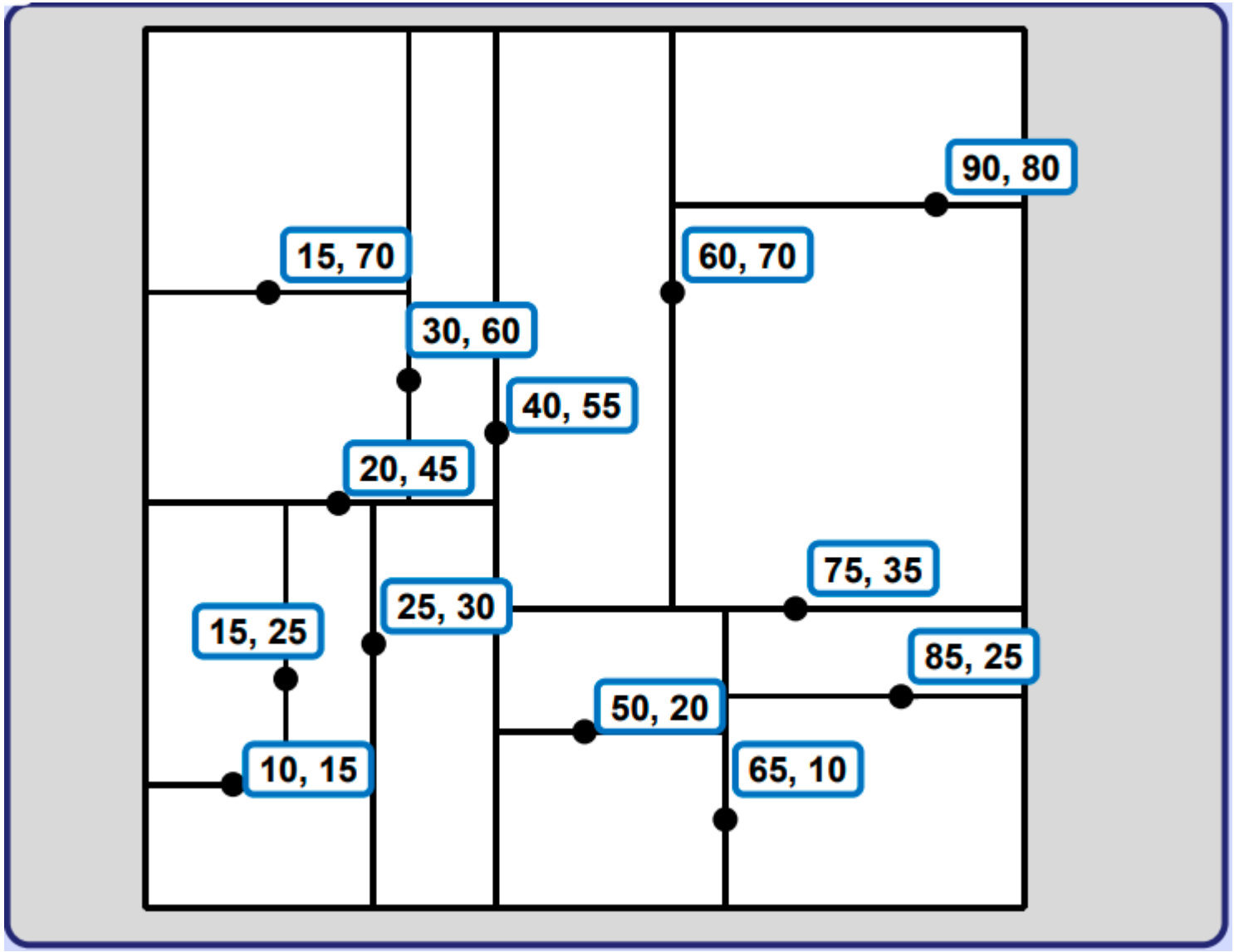



(d) Splitting of region by Sinj

# VÍ DỤ

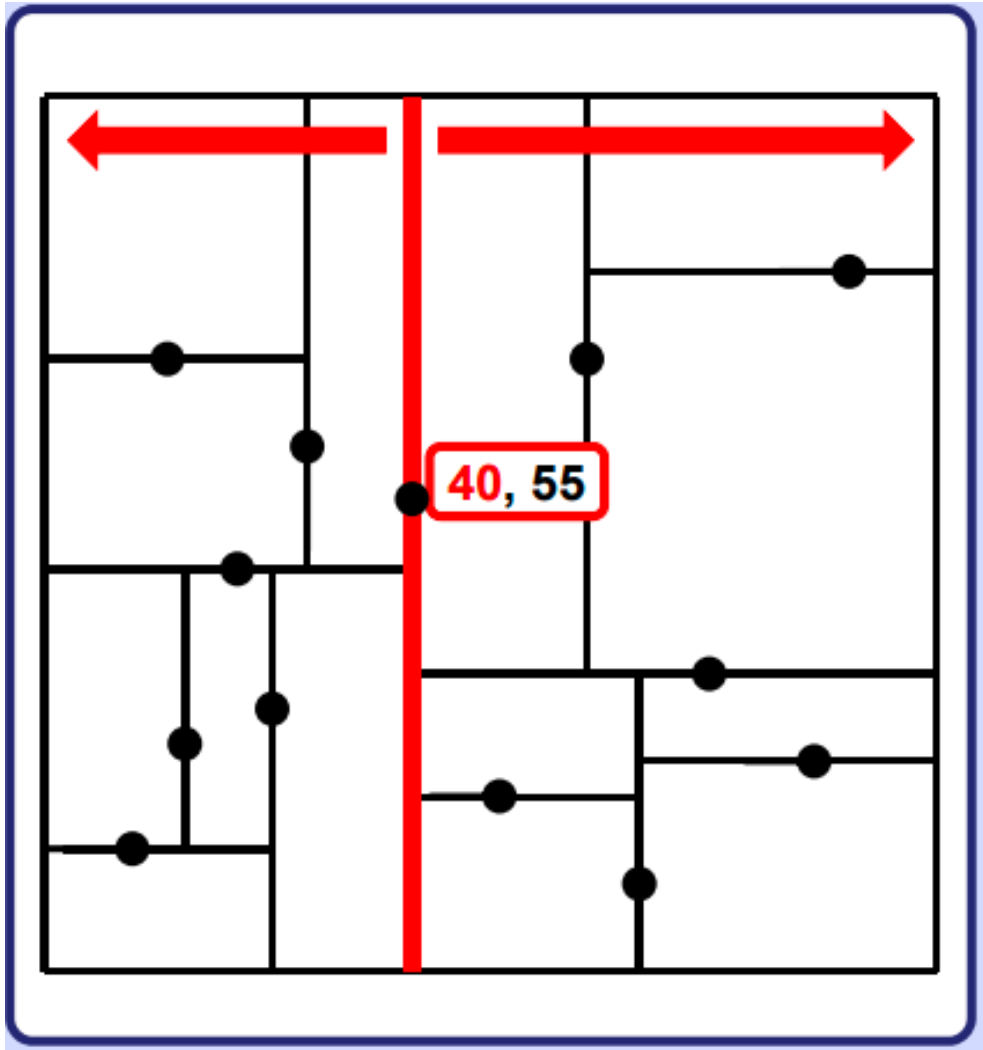

0, 100
100, 100
90, 80
15, 70
60, 70
30, 60
40, 55
20, 45
75, 35
25, 30
15, 25
85, 25
50, 20
10, 15
65, 10
0, 0
100, 0
40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

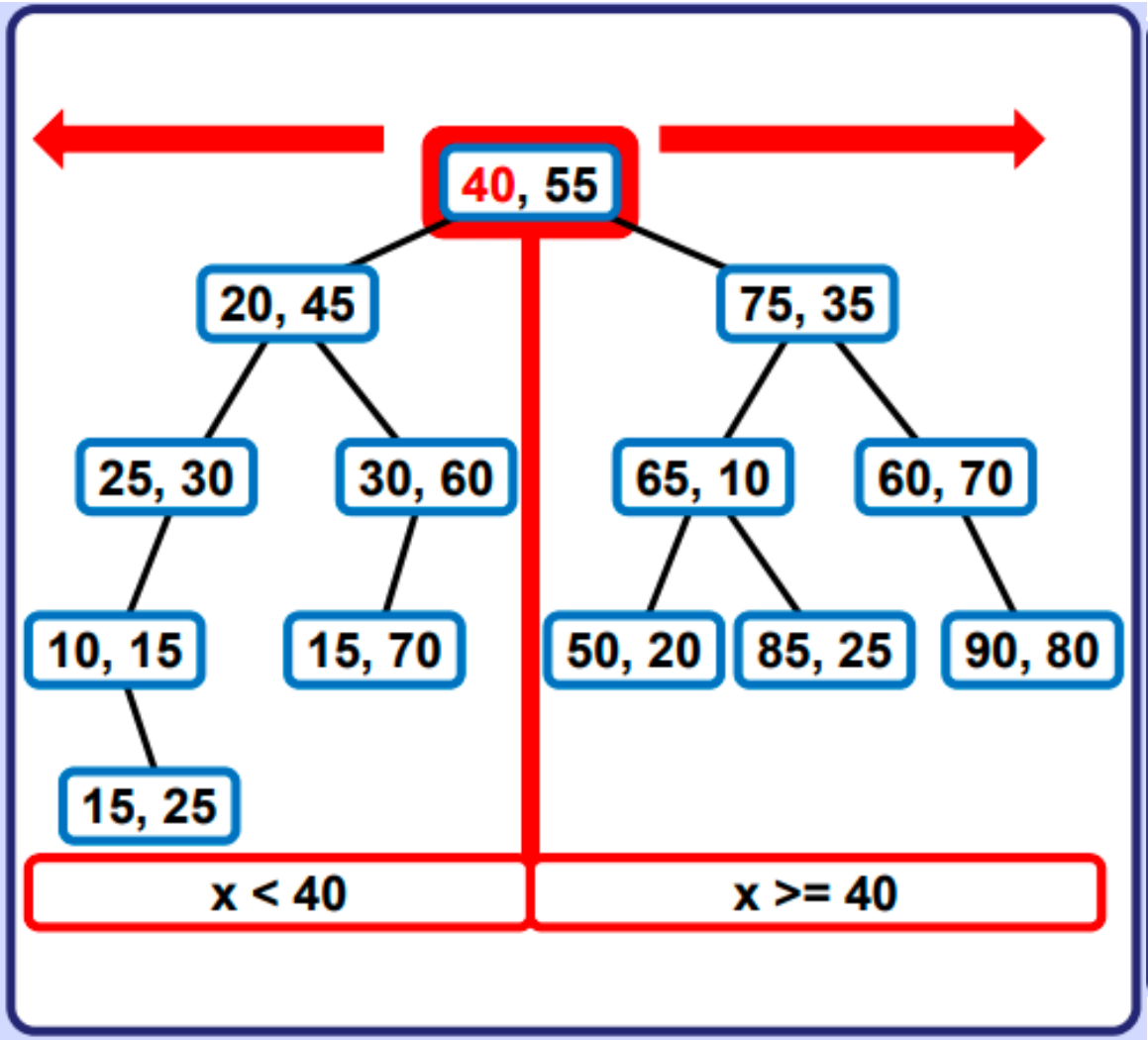
# VÍ DỤ




40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

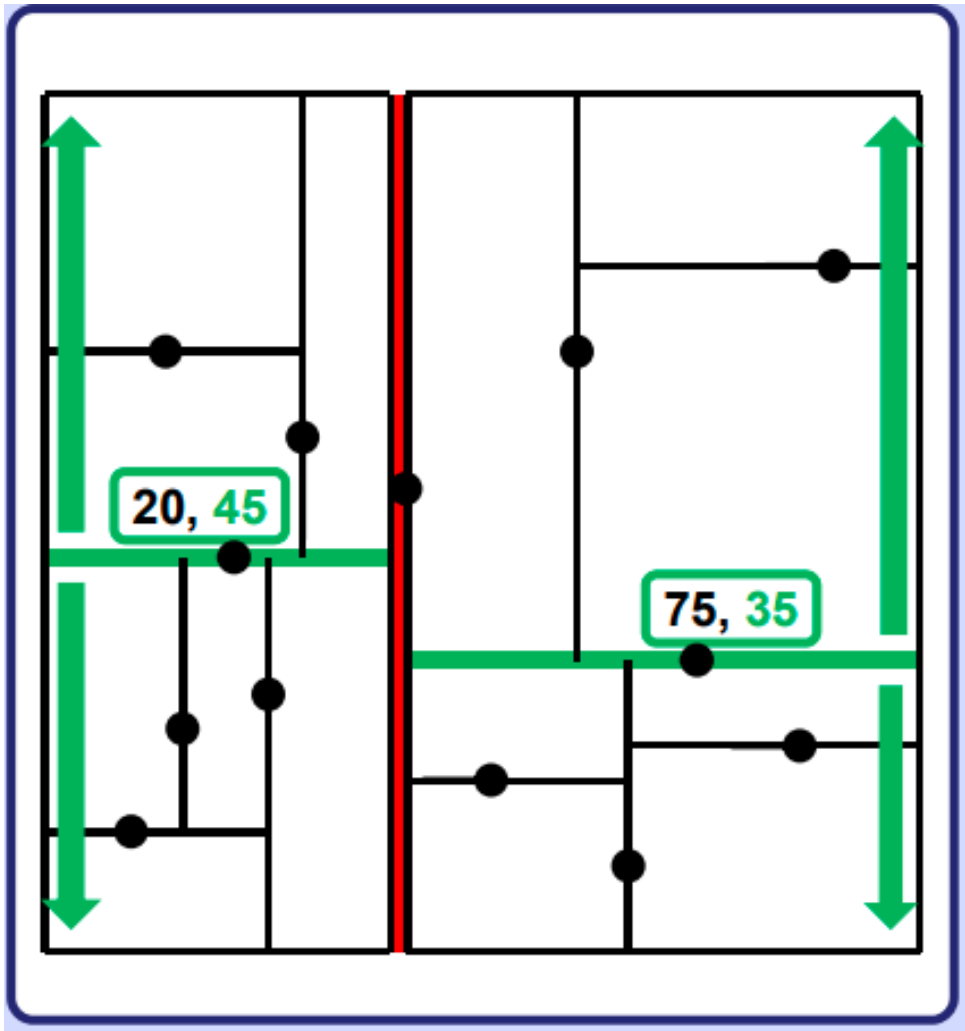
# • VÍ DỤ

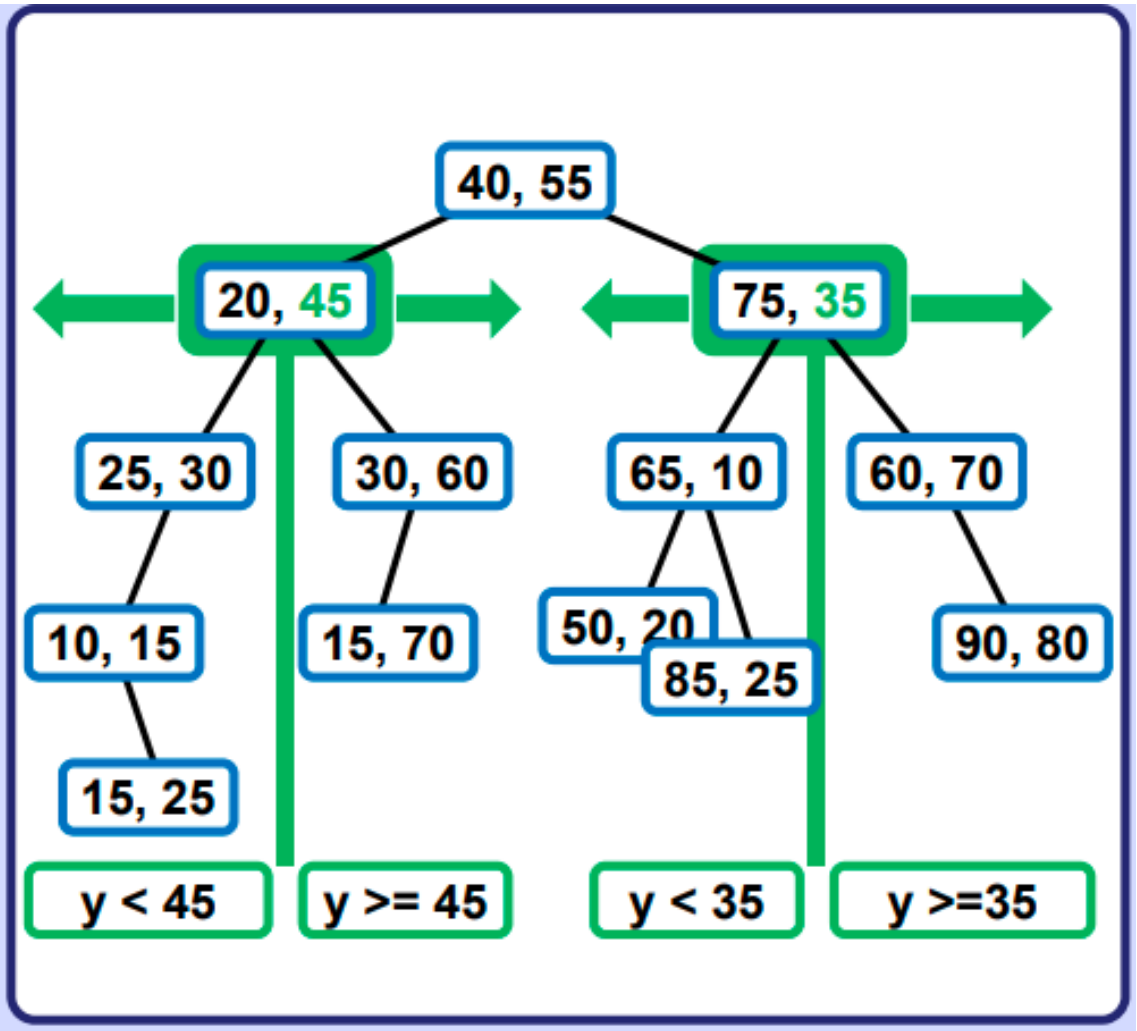



40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

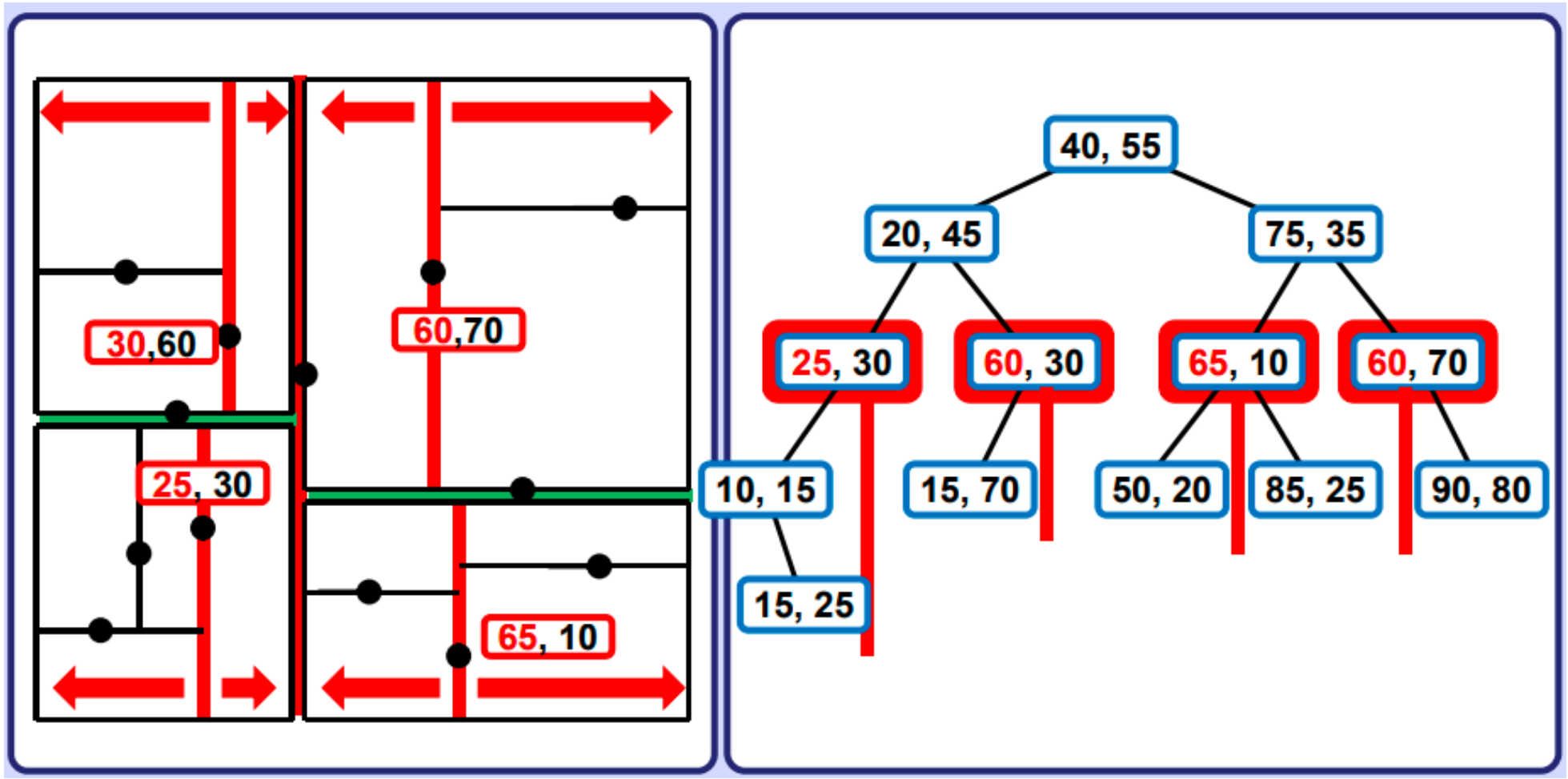
# VÍ DỤ



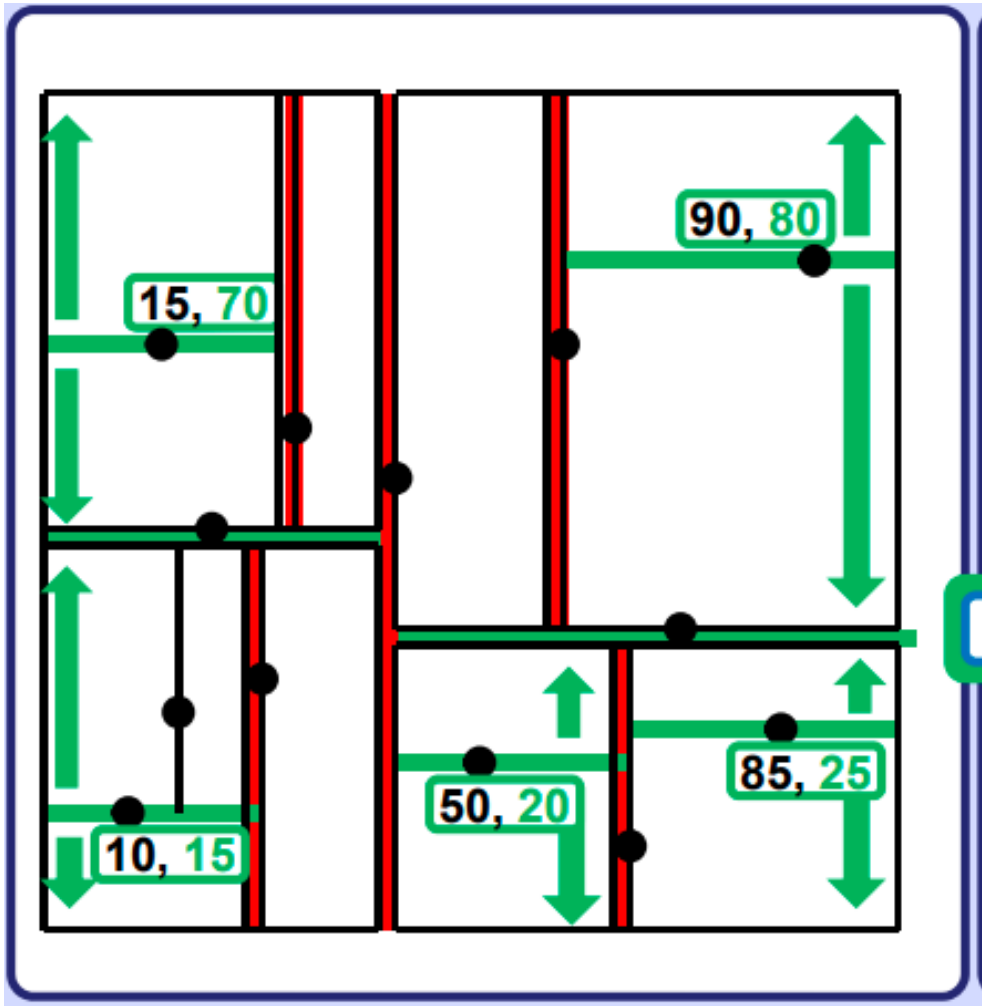



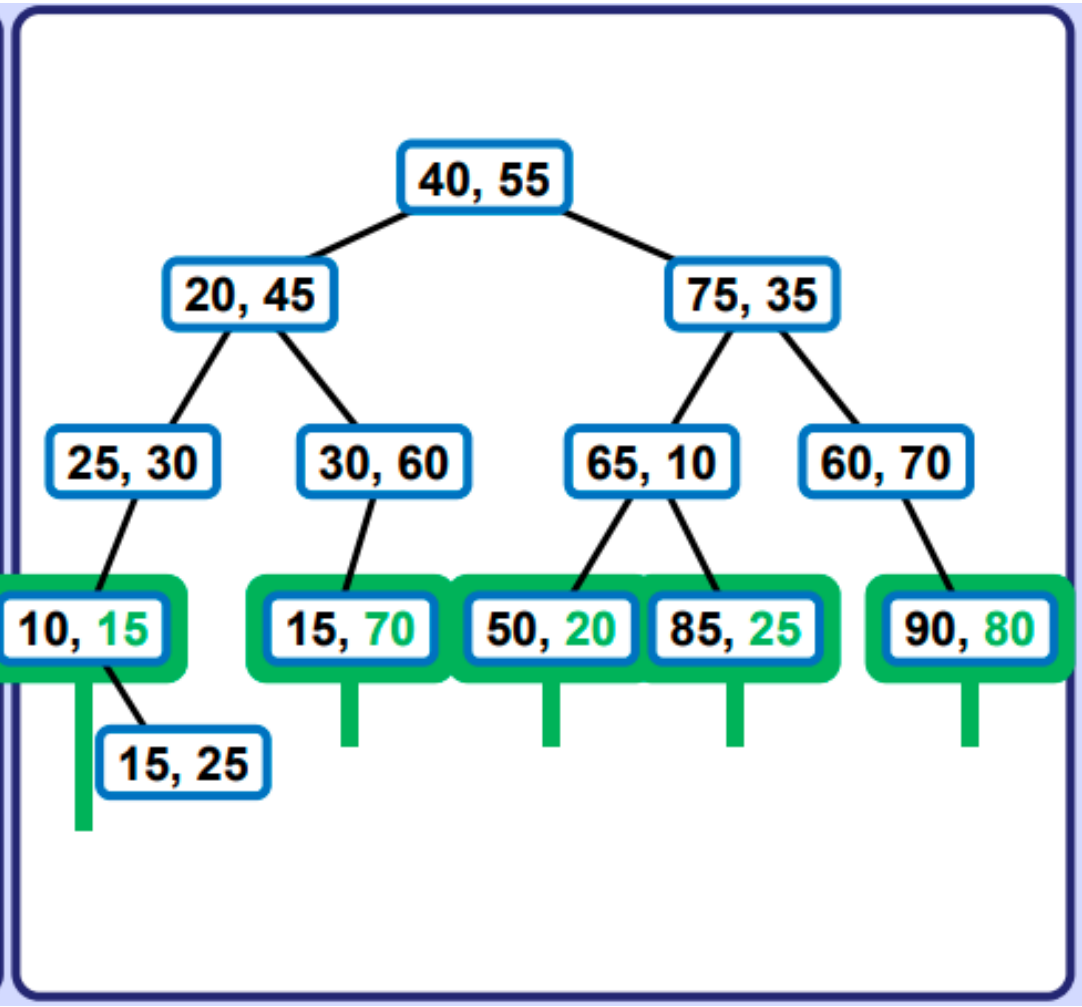

40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

# VÍ DỤ


30,60
60,70
25, 30
65, 10
40, 55
20, 45
75, 35
25, 30
60, 30
65, 10
60, 70
10, 15
15, 70
50, 20
85, 25
90, 80
15, 25
40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

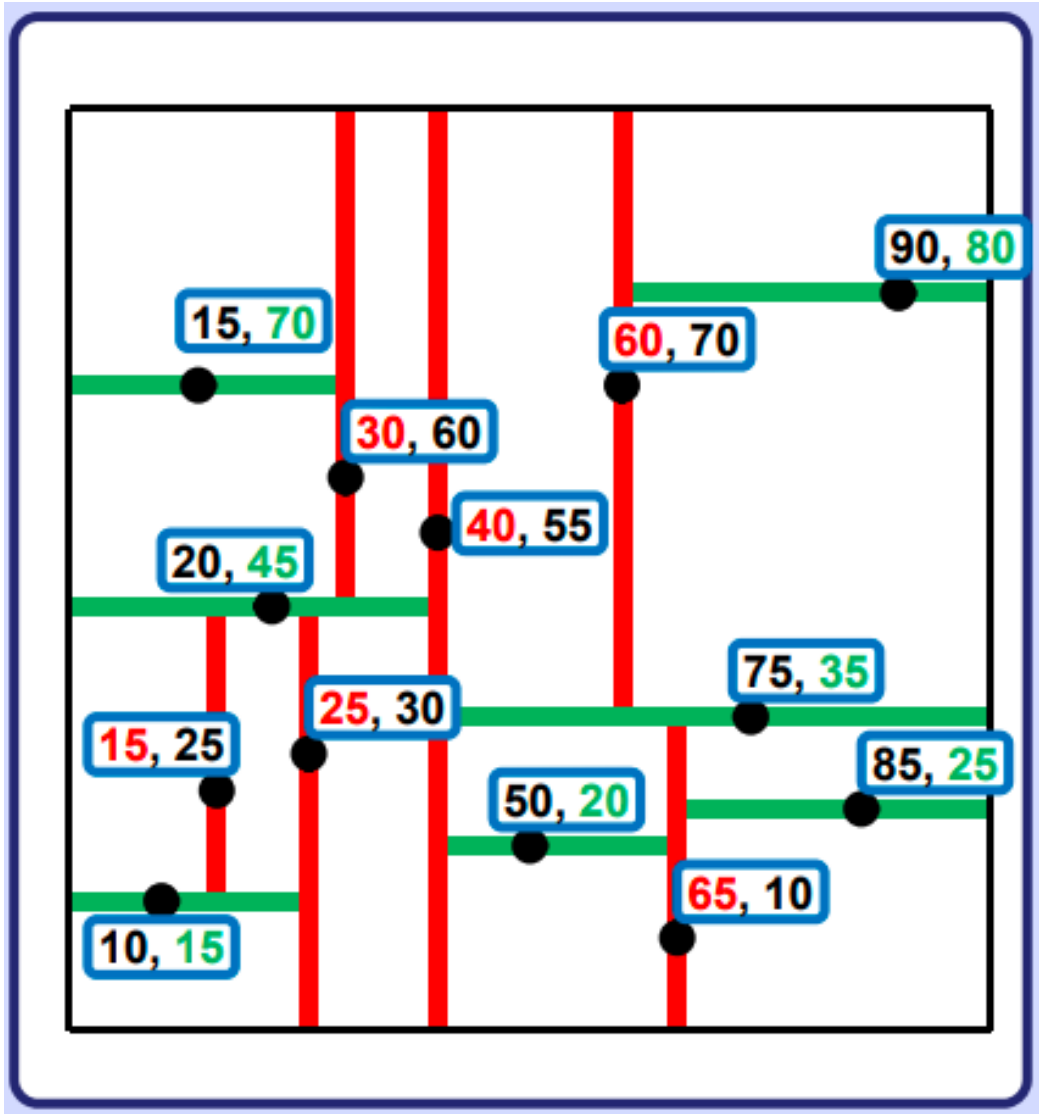
# VÍ DỤ

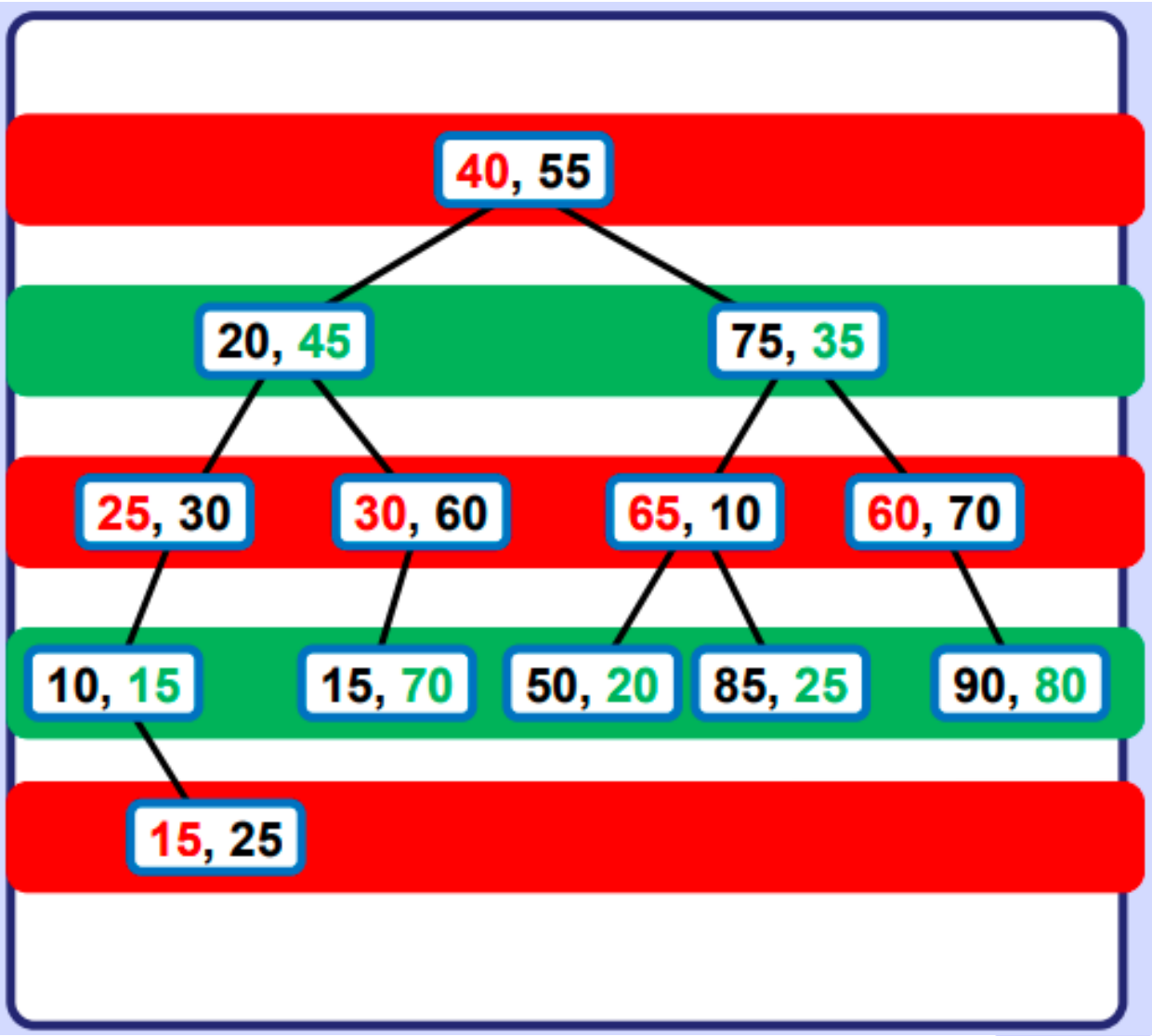



40, 55
20, 45
75, 35
60, 70
30, 60
65, 10
50, 20
15, 70
85, 25
90, 80
15, 25
10, 15
25, 30

# VÍ DỤ


90, 80
15, 70
60, 70
30, 60
40, 55
20, 45
75, 35
25, 30
15, 25
85, 25
50, 20
65, 10
10, 15


40, 55
20, 45
75, 35
25, 30
30, 60
65, 10
60, 70
10, 15
15, 70
50, 20
85, 25
90, 80
15, 25

# Xóa trong 2-D tree

- Gọi T là một 2-D tree
- Nút cần xóa (XVAL, YVAL) = $(x, y)$
- Thuật toán:
  - Tìm N: N.XVAL = $x$ & N.YVAL = $y$
  - Nếu N là nút lá: đặt LLINK/RLINK của cha N về NIL và giải phóng N ⇨ Kết thúc
  - Nếu N là nút trong:
    - ✓Tìm nút thay thế (R) ở trong 2 cây con ($T_l$ và $T_r$)
    - ✓Thay các giá trị không phải con trỏ bằng giá trị của R
    - ✓Lặp để xóa R

# Tìm nút thay thế cho nút bị xóa

- Nếu xóa N ⇨ tìm nút thay thế R: mọi nút thuộc cây con trái (N.LLINK) / phải của N cũng thuộc cây con trái (R.LLINK) / phải tương ứng của R
  - Nếu nút N ở mức chẵn:

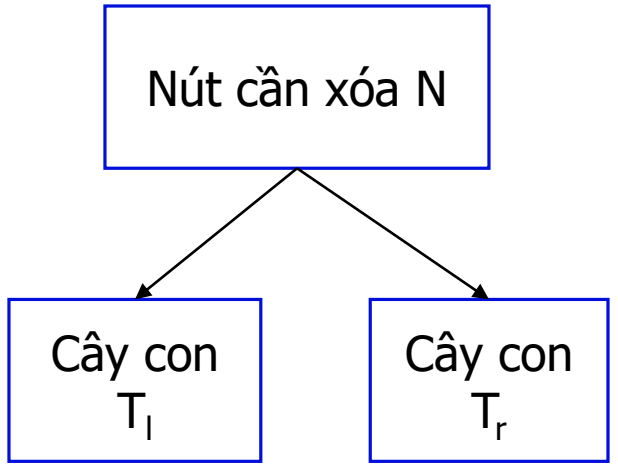
$$\forall M \in N.LLINK: M.XVAL < R.XVAL \;\&$$
$$\forall P \in N.RLINK: P.XVAL \geq R.XVAL$$

  - Nếu nút N ở mức lẻ:

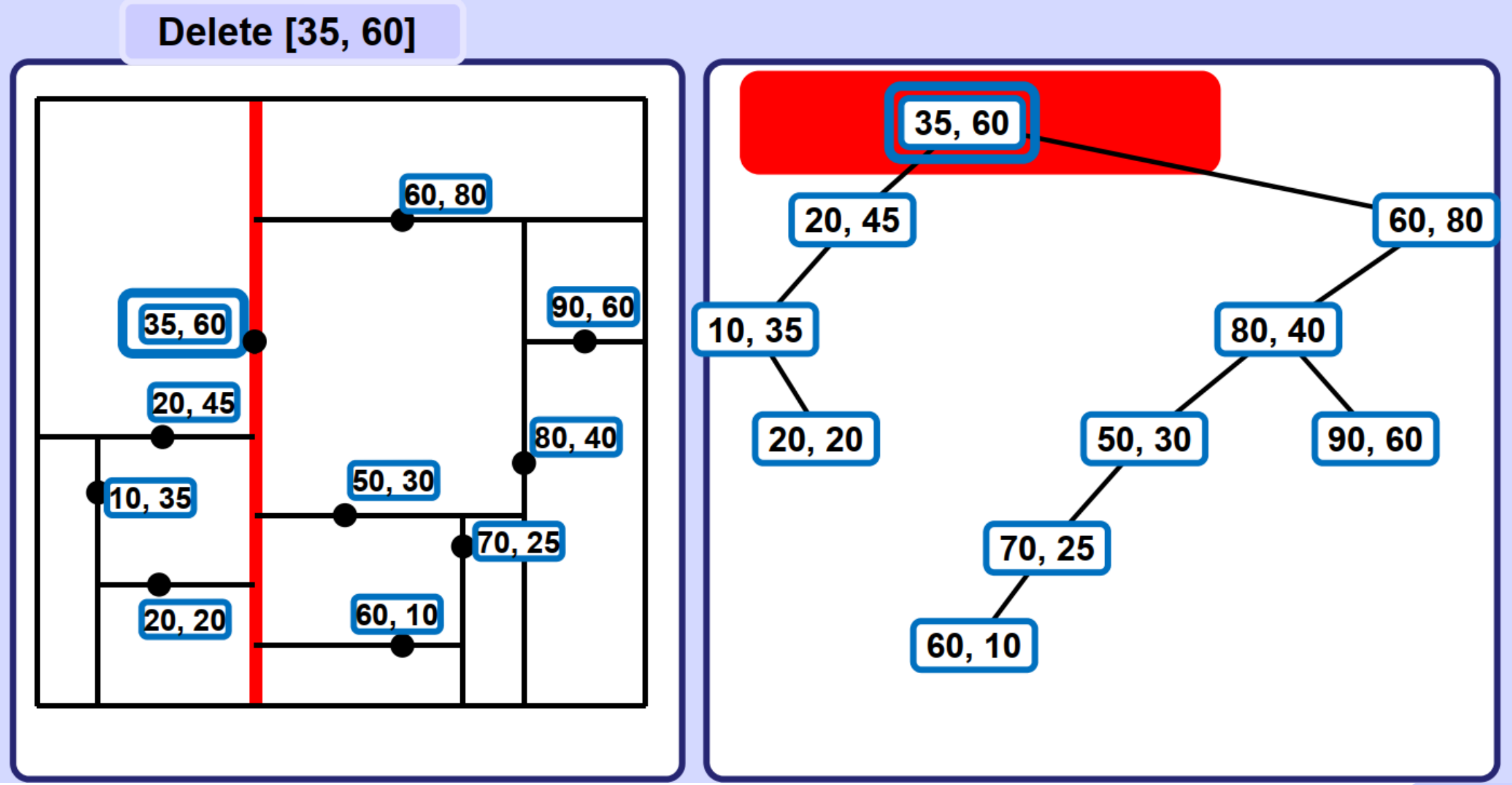
$$\forall M \in N.LLINK: M.YVAL < R.YVAL \;\&$$
$$\forall P \in N.RLINK: P.YVAL \geq R.YVAL$$

# Tìm nút thay thế cho nút bị xóa

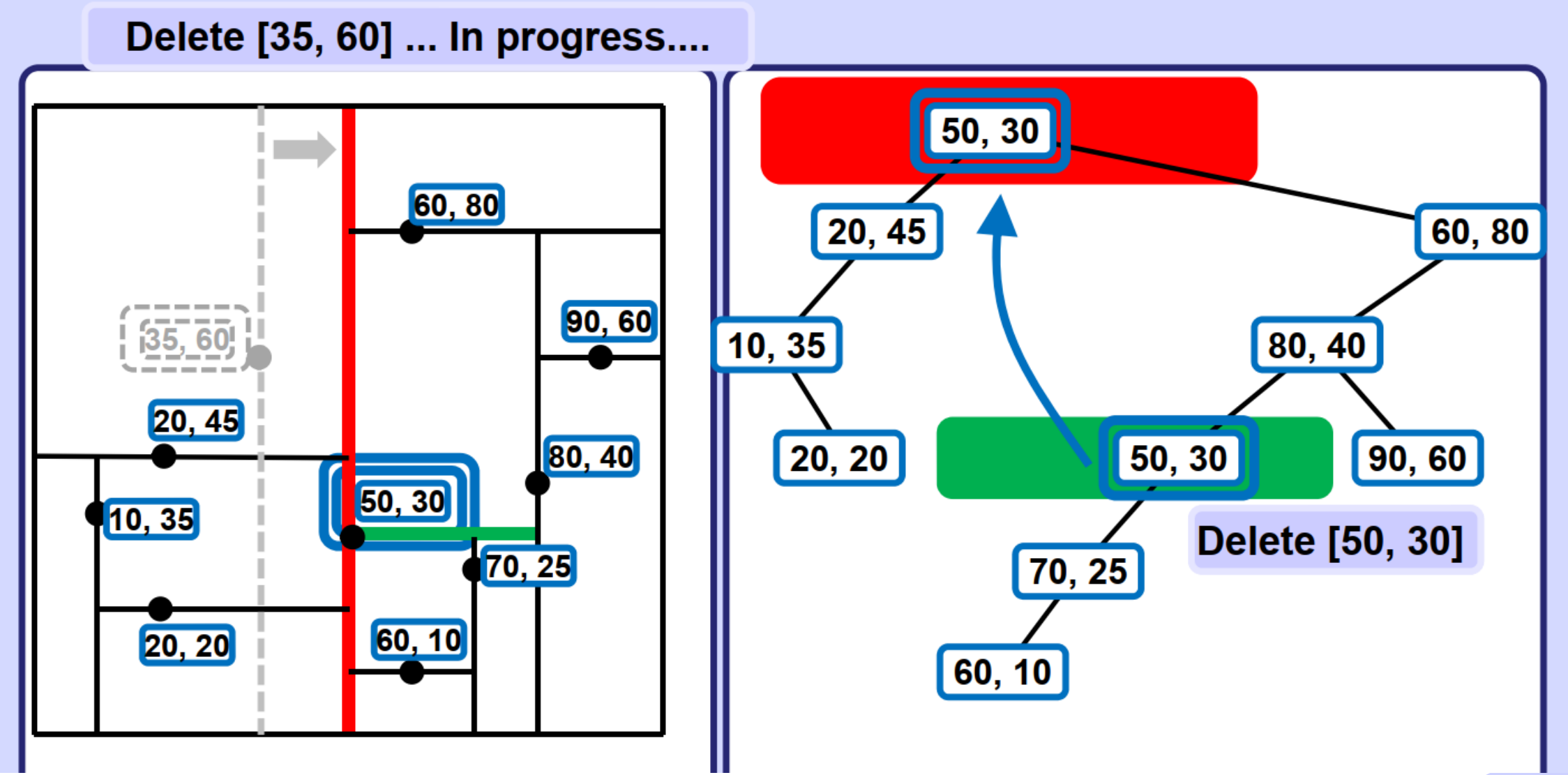
- Nếu N: mức chẵn
  - $T_r$: không rỗng
    - ✓Nút R trong cây con $T_r$ có giá trị XVAL nhỏ nhất là nút thay thế
  - $T_r$: rỗng
    - ✓Tìm nút thay thế bên cây $T_l$
      - Tìm nút R' bên cây trái $T_l$ có XVAL nhỏ nhất
    - ✓N.RLINK = N.LLINK, N.LLINK = NIL
- Nếu N: mức lẻ ⇨ thực hiện tương tự



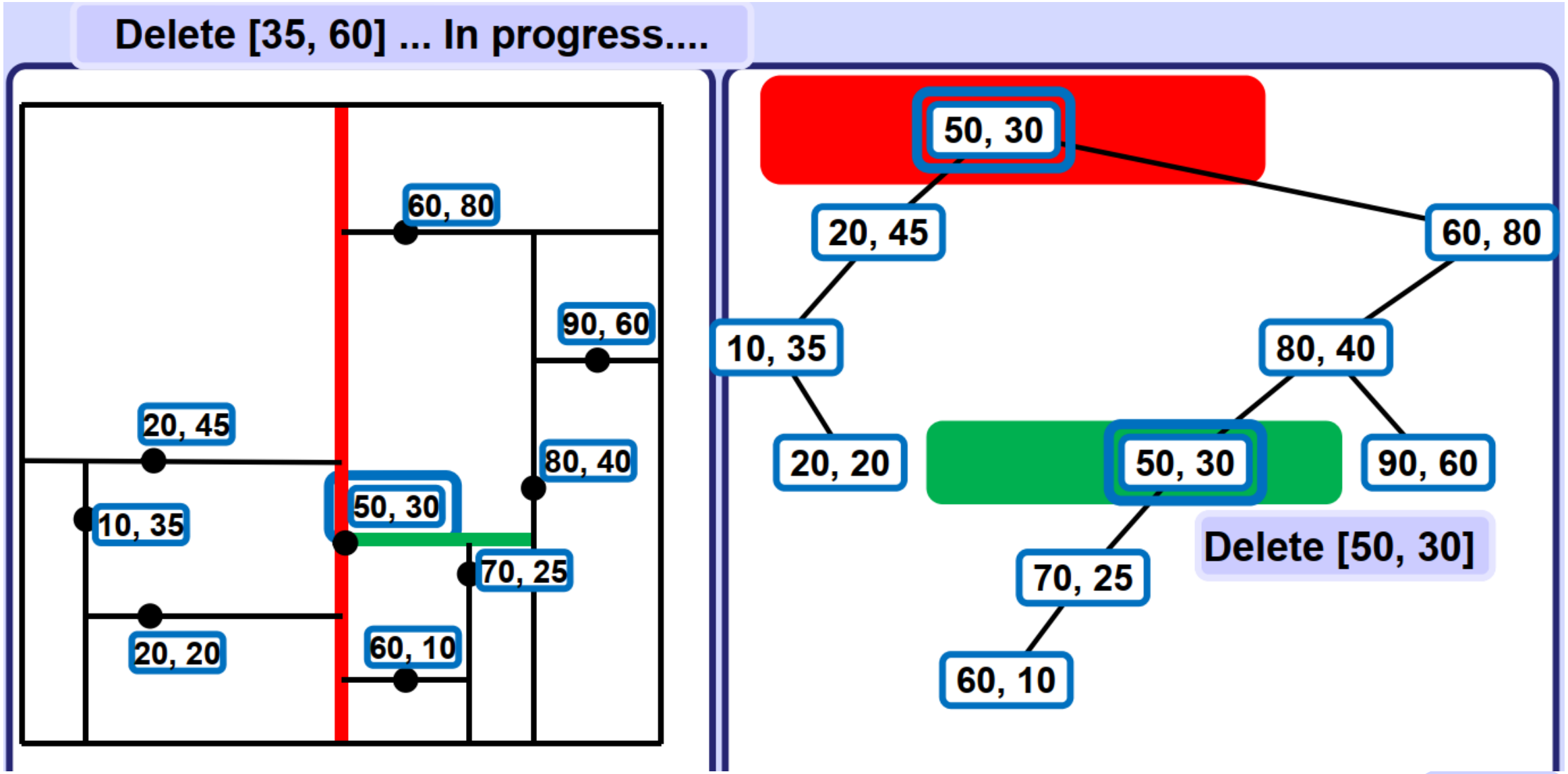
# VÍ DỤ


Delete [35, 60]
60, 80
35, 60
90, 60
20, 45
80, 40
10, 35
50, 30
70, 25
20, 20
60, 10
35, 60
20, 45
60, 80
10, 35
80, 40
20, 20
50, 30
90, 60
70, 25
60, 10

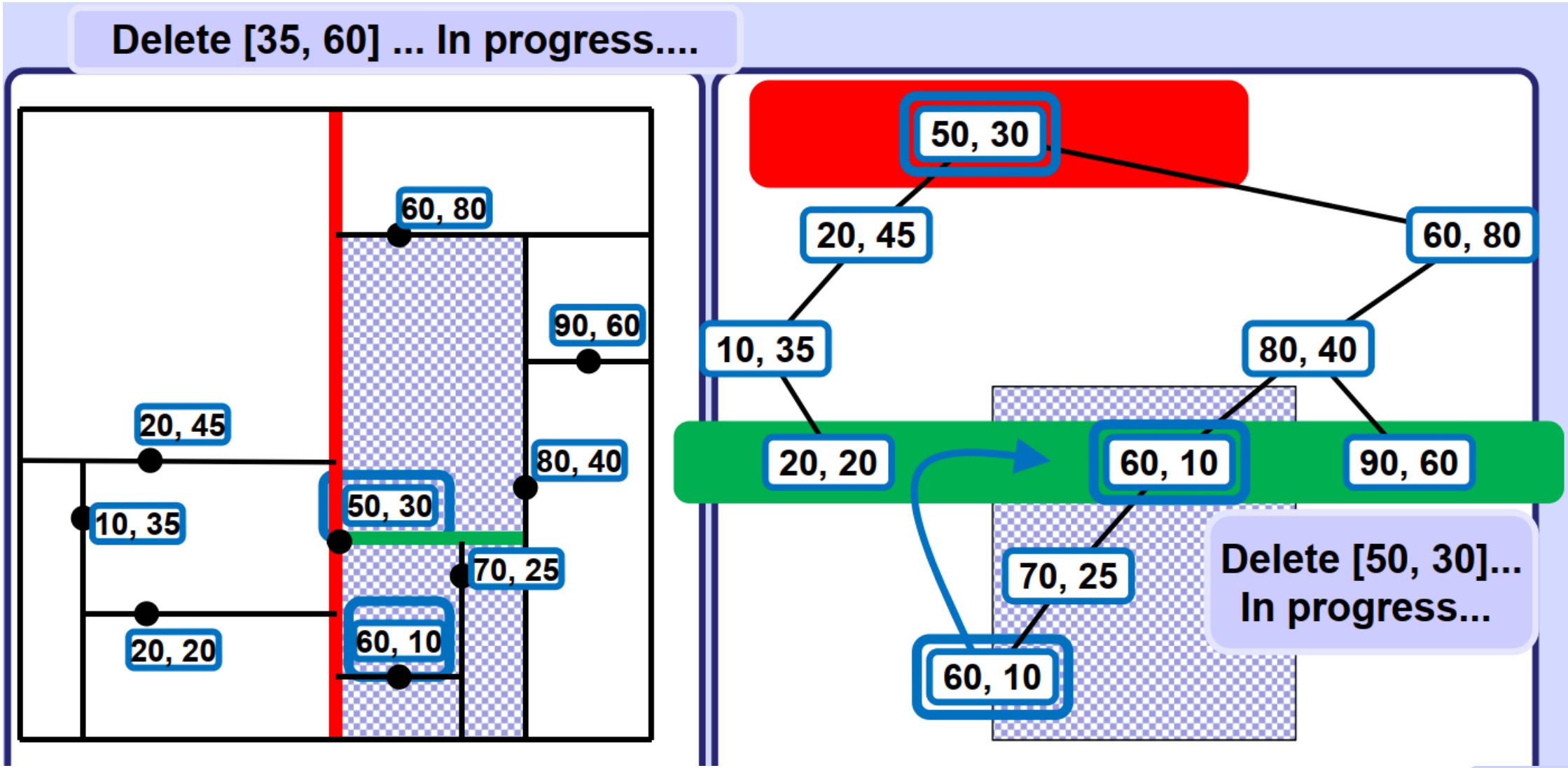
# VÍ DỤ


Delete [35, 60] ... In progress....
60, 80
35, 60
90, 60
20, 45
80, 40
50, 30
10, 35
70, 25
20, 20
60, 10
50, 30
20, 45
60, 80
10, 35
80, 40
20, 20
50, 30
90, 60
Delete [50, 30]
70, 25
60, 10

# VÍ DỤ


Delete [35, 60] ... In progress....
60, 80
90, 60
20, 45
80, 40
50, 30
10, 35
70, 25
20, 20
60, 10
50, 30
20, 45
60, 80
10, 35
80, 40
20, 20
50, 30
90, 60
Delete [50, 30]
70, 25
60, 10

# VÍ DỤ


Delete [35, 60] ... In progress....
60, 80
90, 60
20, 45
80, 40
50, 30
10, 35
70, 25
20, 20
60, 10
50, 30
20, 45
60, 80
10, 35
80, 40
20, 20
60, 10
90, 60
70, 25
60, 10
Delete [50, 30]...
In progress...

# Truy vấn phạm vi trên 2-D tree

- Truy vấn phạm vi: 1 điểm $(x_c, y_c)$ + 1 khoảng cách $r$
- Tìm các điểm $(x, y)$ trên cây 2-D sao cho khoảng cách từ đó đến $(x_c, y_c) \leq r$
  - L1:

    $x_c - r \leq x \leq x_c + r$ $\quad$ $y_c - r \leq y \leq y_c + r$
  - L2 (Euclide)
  - ...

# k-D tree

❖ $p(x_1, x_2, \dots, x_k)$

| INFO | VAL[1] | VAL[2] | … | VAL[k] |
|---|---|---|---|---|
| LLINK | | RLINK | | |

❖ N: 1 nút thuộc k-D tree nếu

➢ Mọi nút M thuộc cây bên trái của N:

$$M.VAL[i] < N.VAL[i]$$

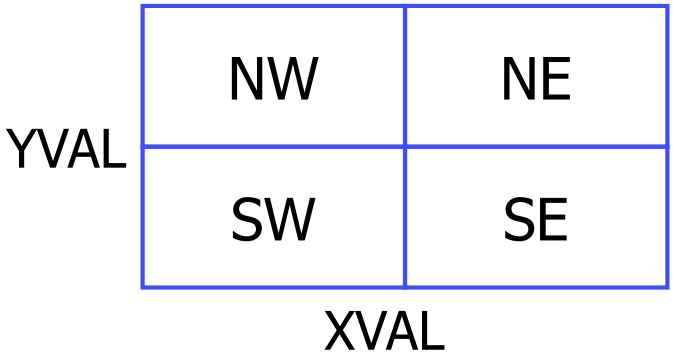
➢ Mọi nút P thuộc cây bên phải của N:

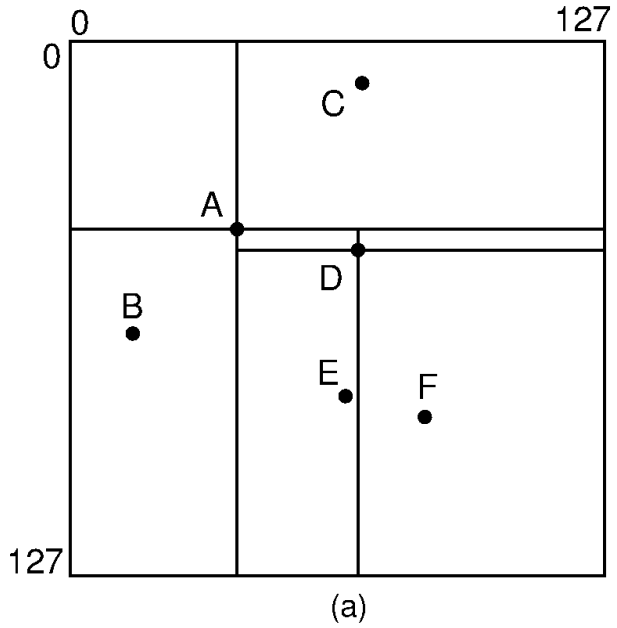
$$P.VAL[i] \geq N.VAL[i]$$

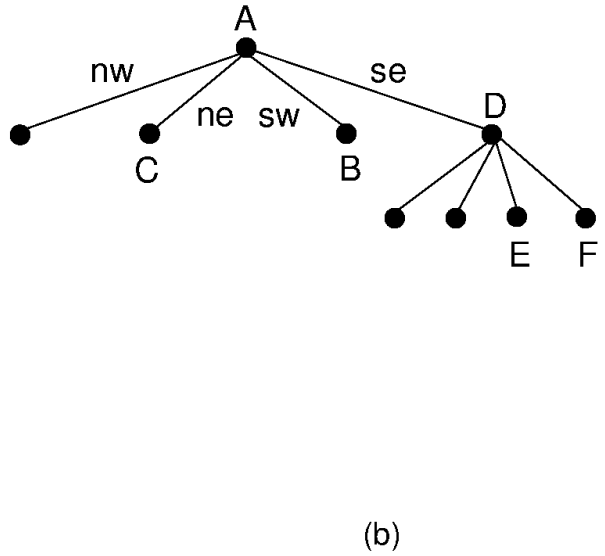
Với $i = level(N) \ mod \ k$

# Cây tứ phân dạng điểm (Point Quadtrees)

- Mỗi điểm trong cây sẽ chia 1 vùng thành 4 vùng con theo cả 2 chiều ngang và dọc (N.XVAL và N.YVAL):
  - NW (Northwest)
  - SW (Southwest)
  - NE (Northeast)
  - SE (Southeast)

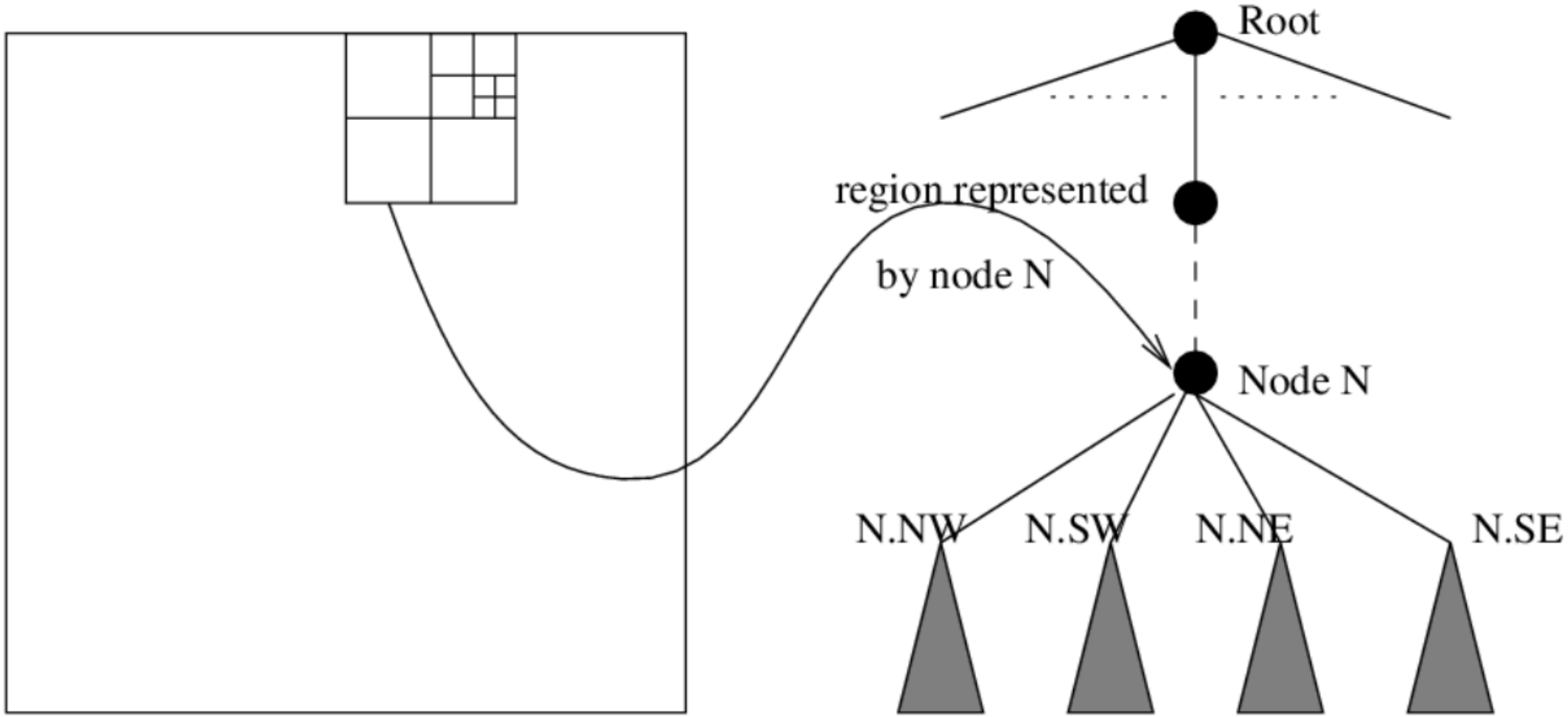


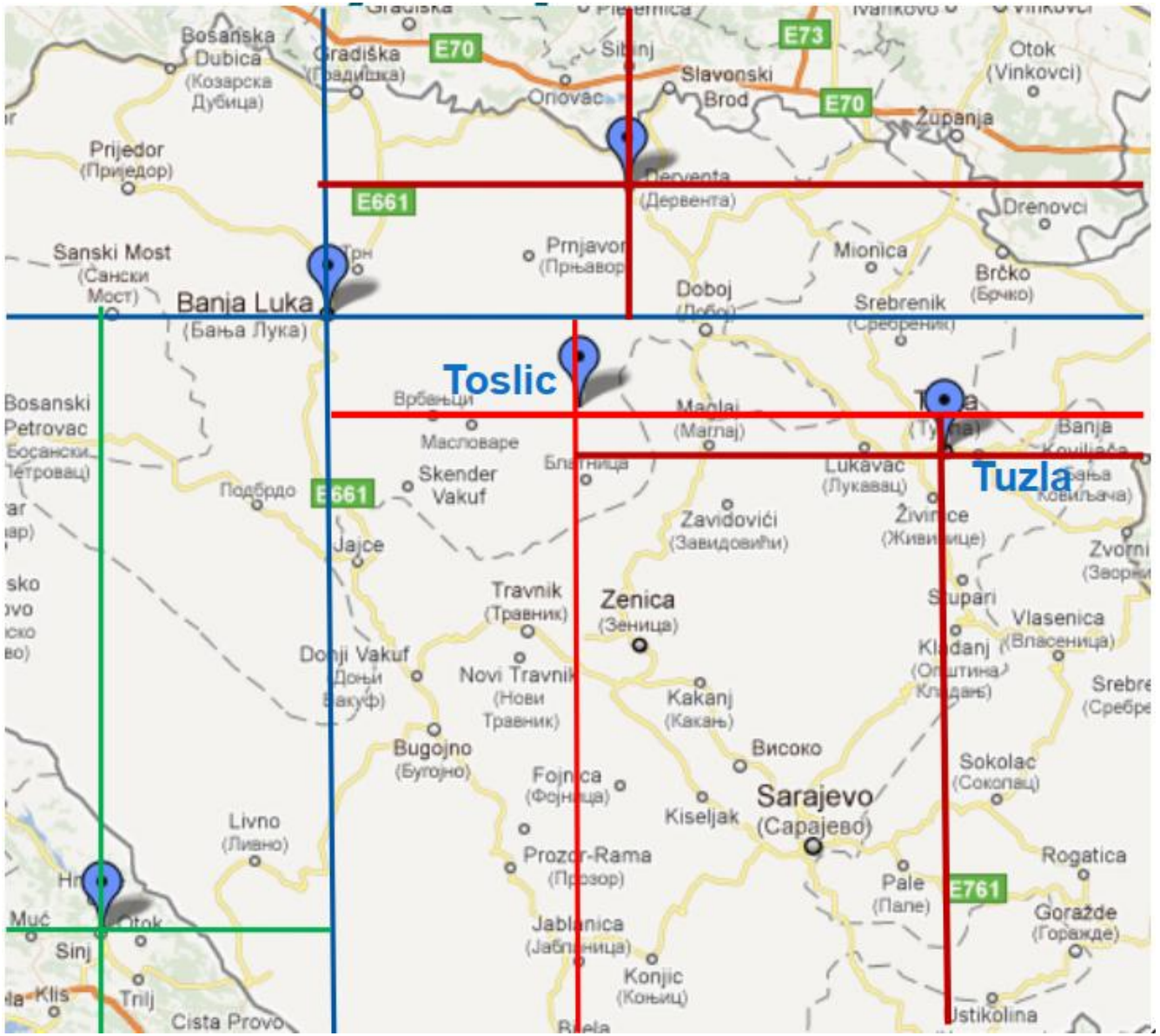



# Cây tứ phân dạng điểm (Point Quadtrees)

- Mỗi nút trong cây tứ phân ngầm biểu diễn 1 vùng:

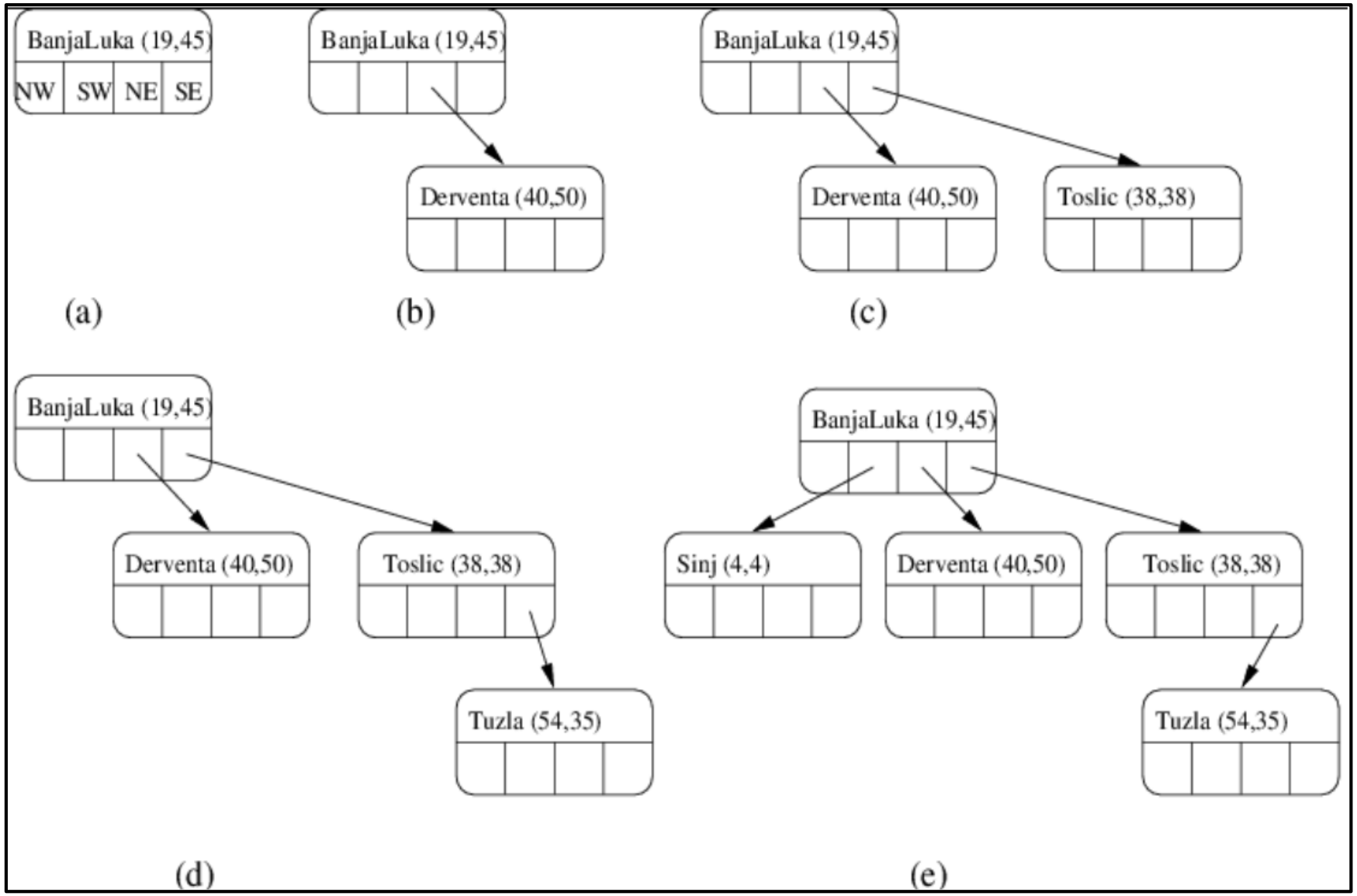


# Thêm dữ liệu vào cây tứ phân




- Banja Luka (19, 45)
- Derventa (40, 50)
- Toslic (38, 38)
- Tuzla (54, 40)
- Sinj (4, 4)

# Thêm dữ liệu vào cây tứ phân



# Xóa dữ liệu trong cây tứ phân

- Nút lá: đơn giản
  - Thiết lập lại con trỏ ở nút cha = NIL và giải phóng nút cần xóa
- Nút trong: phức tạp
  - Cần tìm nút thay thế
  - Ví dụ: xóa nút gốc

# Tìm nút thay thế khi xóa nút trong

- Nút xóa: N
- Nút thay thế R đảm bảo

$$\forall R_1 \in N.NW \Rightarrow R_1 \in R.NW$$
$$\forall R_2 \in N.SW \Rightarrow R_2 \in R.SW$$
$$\forall R_3 \in N.NE \Rightarrow R_3 \in R.NE$$
$$\forall R_4 \in N.SE \Rightarrow R_4 \in R.SE$$

- Ví dụ: N: “Banja Luka” ⇨ R: “Toslic”
- Không phải lúc nào cũng tìm được nút thay thế

# Xóa dữ liệu trong cây tứ phân

- ❖Xóa nút trong ⇨ tìm nút thay thế

⇨chèn lại các nút trỏ bởi NW, SW, NE, SE

⇨ Trường hợp xấu nhất: tất cả các nút bị thay đổi.

# Truy vấn trên cây tứ phân

proc RangeQueryPointQuadtree(T: newqtnodetype, C: circle)

1. If region(T) ∩ C = ∅ then Halt
2. Else
   a) If (T.XVAL, T.YVAL) ∈ C then print(T.XVAL, T.YVAL);
   b) RangeQueryPointQuadtree(T.NW, C);
   c) RangeQueryPointQuadtree(T.SW, C);
   d) RangeQueryPointQuadtree(T.NE, C);
   e) RangeQueryPointQuadtree(T.SE, C);

end proc

# MX-Quadtrees

- Hình dáng cây không phụ thuộc vào
  - Số nút thêm vào
  - Thứ tự thêm vào
- Cho phép xóa và truy vấn hiệu quả

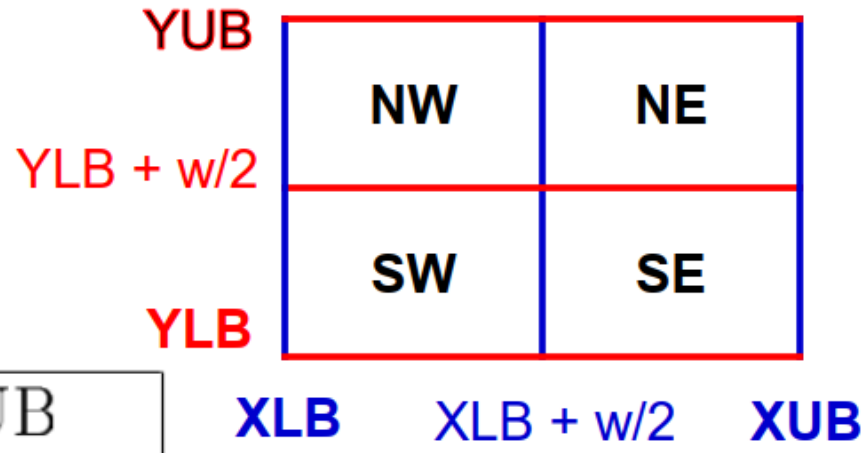
# MX-Quadtrees

- Dữ liệu được chia theo lưới $2^k \times 2^k$
- $k$ tự chọn, nhưng sau khi chọn thì $k$ không được thay đổi
- Cấu trúc nút:
  - Tương tự cây tứ phân dạng điểm
  - Thông tin về vùng biểu diễn (XLB, XUB, YLB, YUB)
  - Nút gốc (root): XLB = 0, XUB = $2^k$, YLB = 0, YUB = $2^k$

# MX-Quadtrees

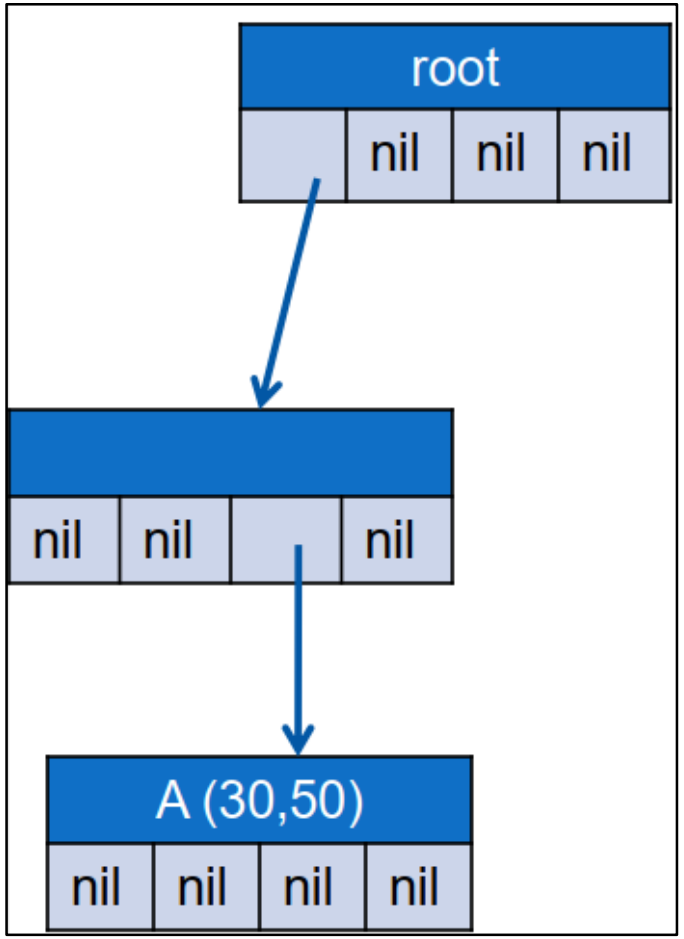
## ❖Cấu trúc nút:

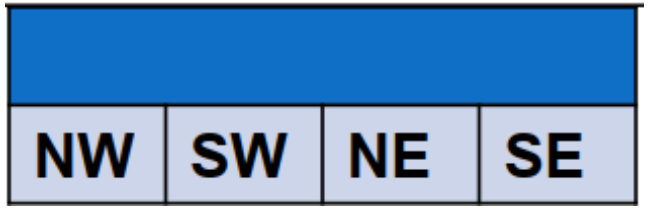
### ➢Các nút con của N (với w = N.XUB – N.XLB)

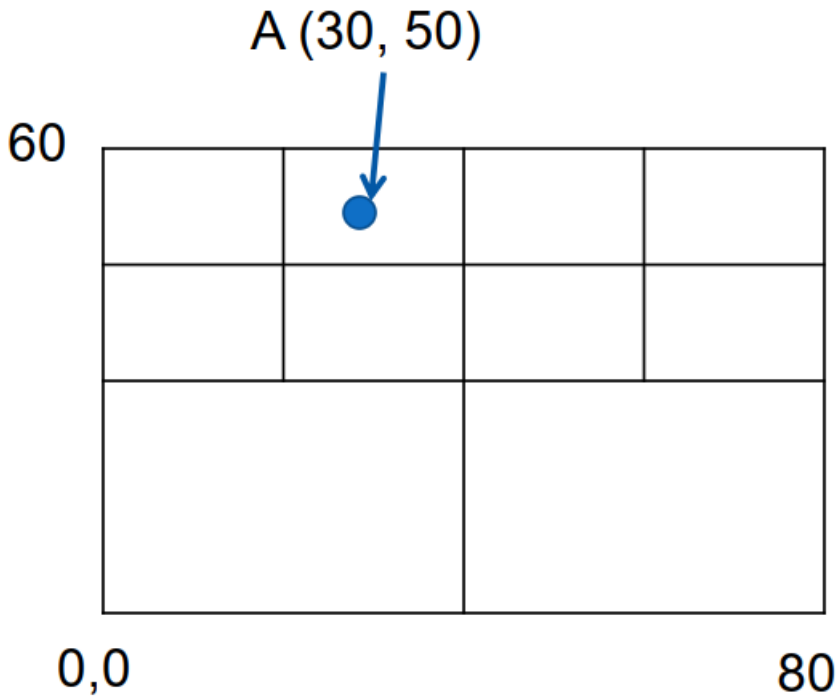



| Child | XLB | XUB | YLB | YUB |
|---|---|---|---|---|
| NW | N.XLB | N.XLB+$\frac{w}{2}$ | N.YLB+$\frac{w}{2}$ | N.YLB+$w$ |
| SW | N.XLB | N.XLB+$\frac{w}{2}$ | N.YLB | N.YLB+$\frac{w}{2}$ |
| NE | N.XLB+$\frac{w}{2}$ | N.XLB+$w$ | N.YLB+$\frac{w}{2}$ | N.YLB+$w$ |
| SE | N.XLB+$\frac{w}{2}$ | N.XLB+$w$ | N.YLB | N.YLB+$\frac{w}{2}$ |

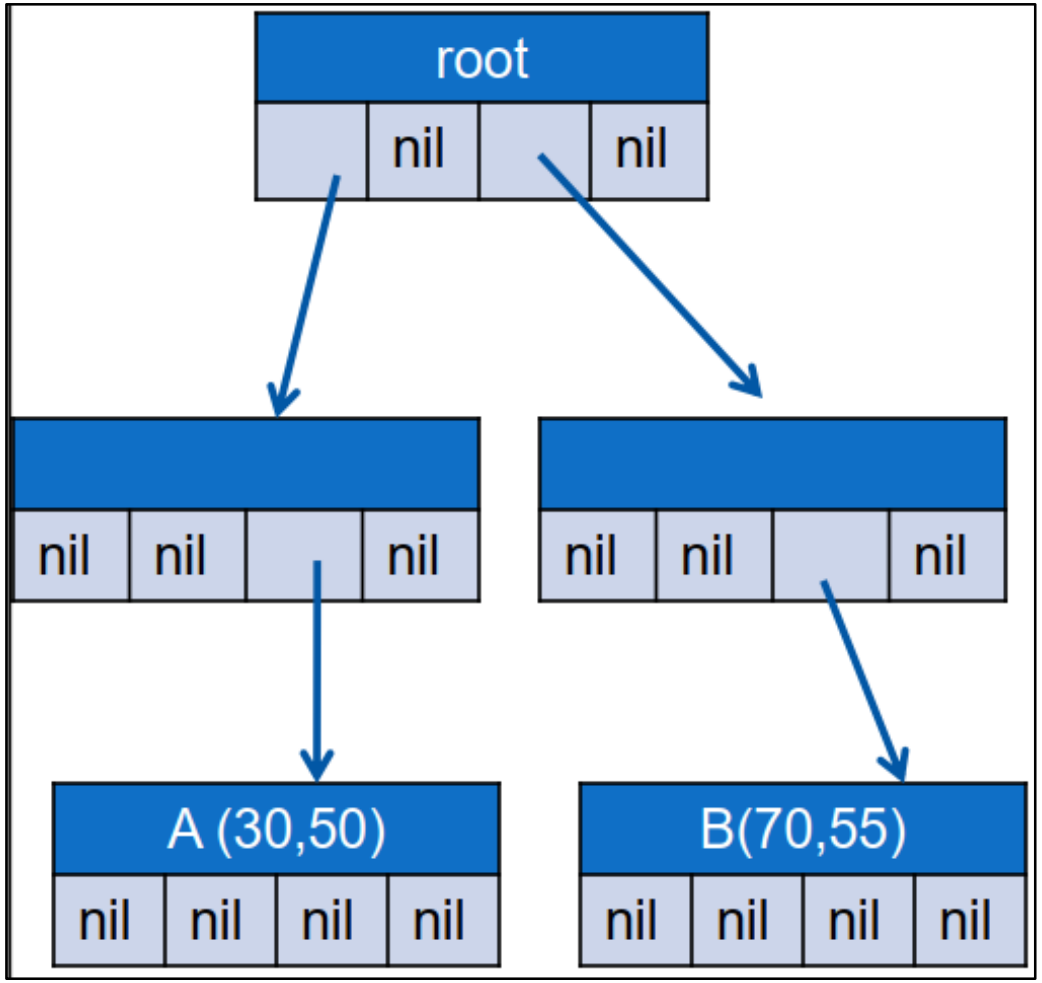
# MX-Quadtrees: Insert

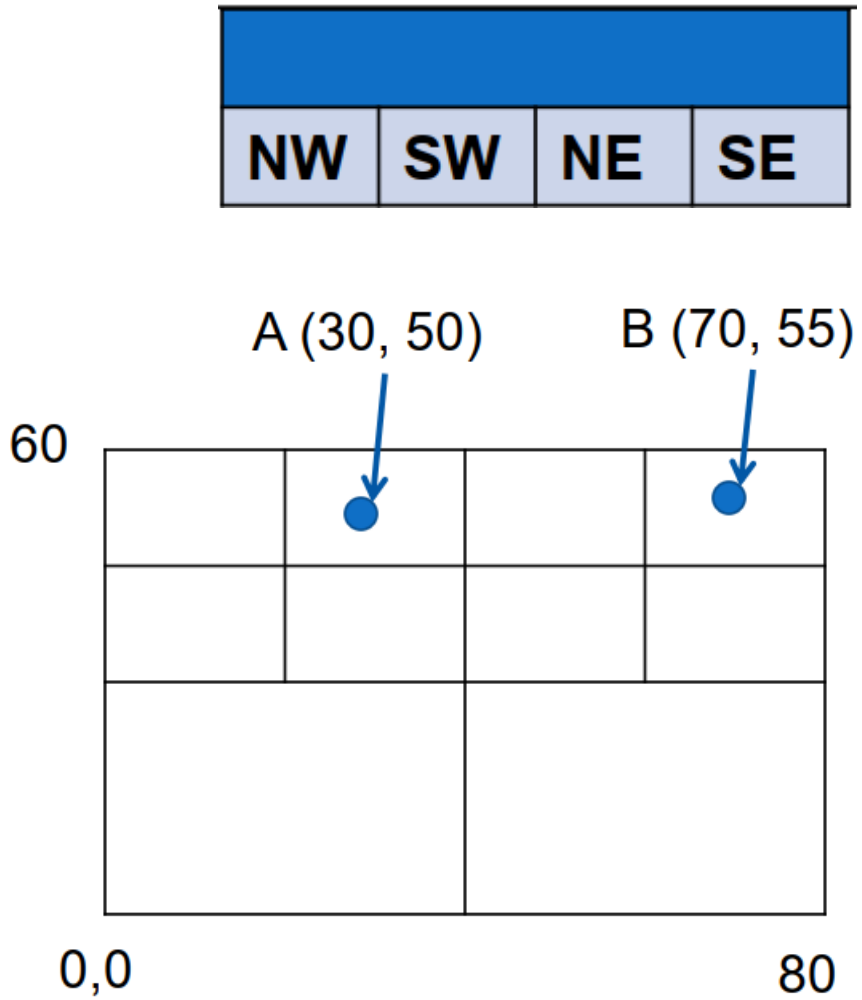


# MX-Quadtrees: Insert

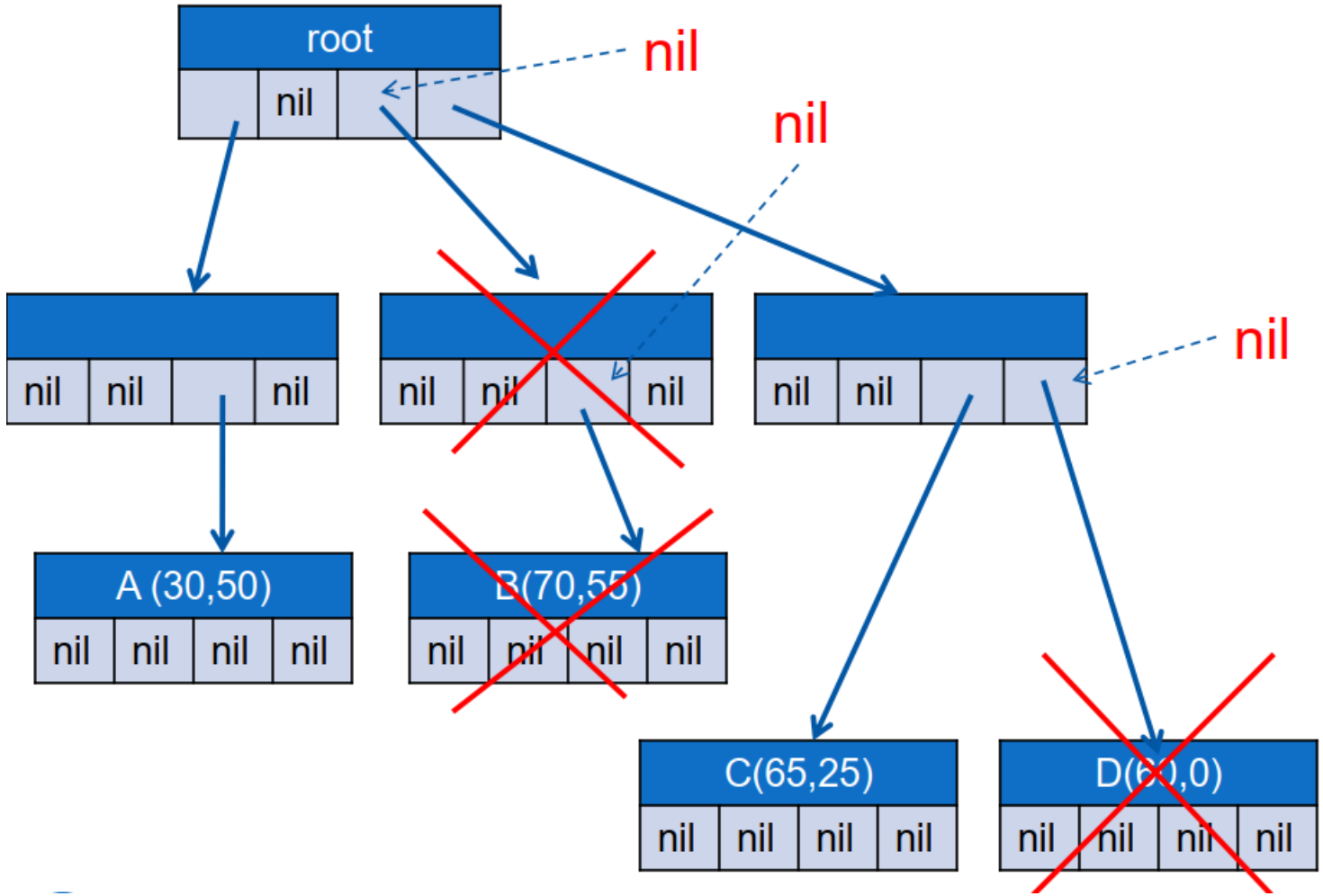


# MX-Quadtrees

- Tất cả các điểm được biểu diễn bởi nút lá
- Nếu N là nút trong của cây MX-Quadtree, thì vùng biểu diễn bởi N chứa ít nhất 1 điểm dữ liệu
- Xóa dễ dàng với độ phức tạp O(k)

# MX-Quadtree: Delete



# MX-Quadtree: Truy vấn phạm vi

- Tương tự cây tứ phân dạng điểm
- Điểm khác biệt: Kiểm tra 1 điểm nằm trong phạm vi truy vấn chỉ thực hiện ở mức lá
- MX-Quadtree:
  - Dư nhiều nút nếu phân bố điểm thưa
  - Nếu nhiều hơn 01 có trong vùng nhỏ nhất
- PR-Quadtree: biến thể của MX-quadtree
  - Nếu 1 nút chỉ có 1 điểm → lưu vào nút đó mà không cần lưu vào nút lá
  - Chỉ phân chia vùng nếu vùng chứa nhiều hơn 1 điểm